

课程编号：(2024-2025-2)-0000550011002-01

课程性质：必修

卫星导航算法与程序设计 1

成果总结报告

学院：_____测绘学院_____

专业：_____导航工程_____

班级：_____23 级智能导航试验一班_____

姓名：_____GYH_____

学号：_____20233021430XX_____

教师：_____王甫红_____

二〇二五年三月三十一日 至 二〇二五年七月三十日

目录

第一章 程序设计概述.....	8
1.1 课程目的	8
1.2 软件介绍	8
1.2.1 总体设计	8
1.2.2 模块设计	8
1.2.3 编程语言	9
1.2.4 软件功能	9
1.3 完成情况	10
第二章 算法与程序设计.....	11
2.1 时间与坐标转换	11
2.1.1 时间转换	11
2.1.1.1 通用时转化为简化儒略日	11
2.1.1.2 简化儒略日转化为通用时	12
2.1.1.3 GPS 时转化为简化儒略日	13
2.1.1.4 简化儒略日转化为 GPS 时	14
2.1.1.5 通用时与 GPS 时相互转换	15
2.1.2 坐标转换	16
2.1.2.1 大地坐标转换为笛卡尔坐标	16
2.1.2.2 笛卡尔坐标转换为大地坐标	17
2.1.2.3 测站地平坐标定位误差计算	18
2.1.2.4 卫星的高度角与方位角计算	19
2.2 NovATel 数据获取	21
2.2.1 数据格式	21
2.2.2 解码函数	22
2.2.2.1 CRC 校验	22
2.2.2.2 解码主函数	22
2.2.3 字节提取	23

2.2.4 观测数据提取	23
2.2.4.1 观测数据提取	23
2.2.4.2 GPS 广播星历提取	24
2.2.4.3 BDS 广播星历提取	26
2.2.4.4 定位结果提取	26
2.3 卫星位置计算	30
2.3.1 GPS 卫星	30
2.3.1.1 卫星位置与速度	30
2.3.1.2 卫星钟差与钟速	32
2.3.2 BDS 卫星	34
2.3.2.1 卫星位置与速度	34
2.3.2.2 卫星钟差与钟速	35
2.3.3 过期星历	35
2.4 误差改正	37
2.4.1 大气误差	37
2.4.1.1 对流层延迟	37
2.4.1.2 电离层延迟	38
2.4.2 粗差探测	38
2.5 单点定位与测速	40
2.5.1 信号发射时刻卫星状态	40
2.5.2 单点定位	42
2.5.3 单点测速	44
2.6 获取实时数据流	47
2.7 主函数设计	48
2.7.1 实现逻辑	48
2.7.2 C++实现	49
第三章 结果精度分析	50
3.1 引言	50

3.2 数据说明	50
3.2.1 数据来源.....	50
3.2.2 结果格式	51
3.3 结果分析	52
3.3.1 结果展示.....	52
3.3.2 结果分析.....	56
3.3.2.1 定位误差分量的系统性特征分析.....	56
3.3.2.2 接收机钟差对定位误差的分析.....	58
3.3.2.3 其他残余误差源评估.....	59
3.3.2.4 误差分析总结与改进.....	59
第四章 体会与心得.....	61
参考文献.....	63

表目录

表 1-2-1 模块设计说明表格.....	8
表 2-1-1 通用时转化为简化儒略日函数功能与说明表格.....	12
表 2-1-2 简化儒略日转化为通用时函数功能与说明表格.....	13
表 2-1-3 GPS 时转化为简化儒略日函数功能与说明表格	14
表 2-1-4 简化儒略日转化为 GPS 函数功能与说明表格	15
表 2-1-5 通用时与 GPS 时相互转化函数功能与说明表格	15
表 2-1-6 大地坐标转化为笛卡尔坐标函数功能与说明表格.....	17
表 2-1-7 笛卡尔坐标转化为大地坐标函数功能与说明表格.....	18
表 2-1-8 测站地平坐标定位误差计算函数说明表格.....	19
表 2-1-9 计算卫星高度角与方位角函数说明表格.....	20
表 2-2-1 OEM 7 的消息的三个重要部分表格.....	21
表 2-2-2 常见的 Message ID 类型以及内容表格	21
表 2-2-3 字节提取函数总列表格.....	23
表 2-2-4 观测数据提取函数的说明表格.....	24
表 2-2-5 GPS 卫星星历提取函数的说明表格	26
表 2-2-6 BDS 卫星星历提取函数的说明表格.....	26
表 2-2-7 定位结果提取的函数说明表格.....	27
表 2-3-1 GPS 卫星位置速度、钟差钟速的计算函数说明表格	33
表 2-3-2 BDS 卫星位置速度、钟差钟速的计算函数说明表格.....	35
表 2-3-3 过期星历判断的函数说明表格.....	36
表 2-4-1 对流层延迟误差计算的函数说明表格.....	38
表 2-4-2 粗差探测的计算函数说明表格.....	39

表 2-5-1 信号发射时刻卫星状态计算的函数说明表格.....	41
表 2-5-2 单点定位结果计算的函数说明表格.....	44
表 2-5-3 单点测速结果的计算函数说明表格.....	46
表 2-6-1 获取实时数据流的函数说明表格.....	47
表 3-2-1 数据采集时间段说明表格.....	50
表 3-2-2 定位结果误差格式说明表格.....	51
表 3-3-1 12 小时定位误差统计表格.....	52
表 3-3-2 12 小时定位精度统计表格.....	53
表 3-3-3 12 小时定位接收机钟差与钟漂统计表格.....	54
表 3-3-4 12 小时定位时卫星使用数量统计表格.....	55
表 3-3-5 本项目 SPP 误差源分析综合表格.....	60

图目录

图 2-1-1 通用时、GPS 时、儒略日与年积日相互转换整体思路图	11
图 2-1-2 大地坐标、笛卡尔坐标与 ENU 坐标相互转换整体思路图	16
图 2-2-1 观测数据格式（上）卫星星历（中）定位结果（下）	29
图 2-3-1 轨道平面坐标系（左）轨道参数（右）	30
图 2-5-1 单点定位与测速的基本原理图	40
图 3-2-1 GNSS 板卡 司南 K708（左） 芯星通 UB480（右）	50
图 3-2-2 示例数据格式	51

第一章 程序设计概述

1.1 课程目的

用标准 C/C++语言，在 PC 平台下实现从 OEM719 或 K708、UB4B0、GNSS 板卡中实时获取伪距、多普勒频移、广播星历等数据，进行导航定位测速（PVT）解算，并在 Windows 控制台中输出实时定位结果与精度统计。

1.2 软件介绍

1.2.1 总体设计

本课程设计的软件整体功能以及总体结构组成如下：

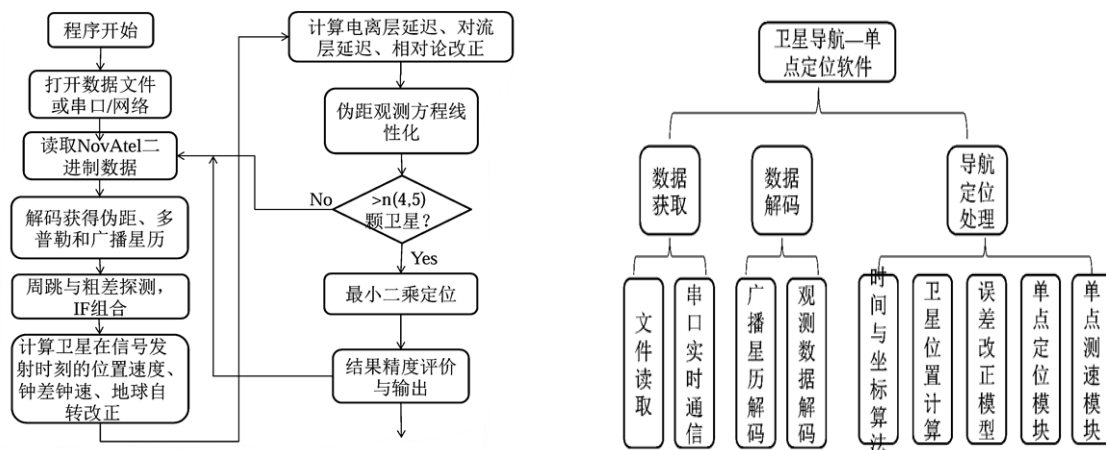


图 1-2-1 软件整体工作流程（左） 软件总体结构（右）

1.2.2 模块设计

本软件针对每个模块，均设计了相对独立的、具有一定功能的子函数，其中设计方法与实现步骤，也是本文在第二章中“算法与程序设计”的说明结构，如下表格：

表 1-2-1 模块设计说明表格

顺序	模块设计内容	说明
1	模块结构	各功能模块的组织方式和流程结构
2	数学原理	所以来的核心计算原理与公式
3	结构体定义	数据结构的组织与字段的含义
4	函数介绍	关键函数及其输入输出说明
5	结果输出/保存	该模块最终输出的格式与保存方式

1.2.3 编程语言

1. 开发环境:

本软件在 Microsoft Visual Studio 2022 集成开发环境中，使用 C++语言进行开发。

2. 编程规范:

为保证代码的专业性、可读性和可维护性，整个项目严格遵守了课程中强调的 C++编码与命名等规则，具体规则如下：

- 1) 命名规则：采用函数名大驼峰命名法，变量名采用小驼峰或下划线分割法；
- 2) 变量规则：多个变量在传输的时候，使用数组/向量，而非逐个传递；
- 3) 注释规则：对关键函数、参数、复杂逻辑和数据结构进行详细地注释。

1.2.4 软件功能

为实现课程目标，本软件的功能与技术规格定义如下：

1. 输入:

- 1) GNSS 系统与信号：软件设计支持处理 GPS 与 BDS 双系统的双频信号，具体为 GPS 的 L1/L2 频点，与 BDS 的 B1/B3 频点；
- 2) 数据内容：软件能够解析和处理的核心数据包括两大系统的广播星历与观测数据等；
- 3) 数据获取模式：软件支持两种数据获取模式：

[1] 文件模式：从预先存储的 NovAtel 二进制格式数据文件中读取数据；

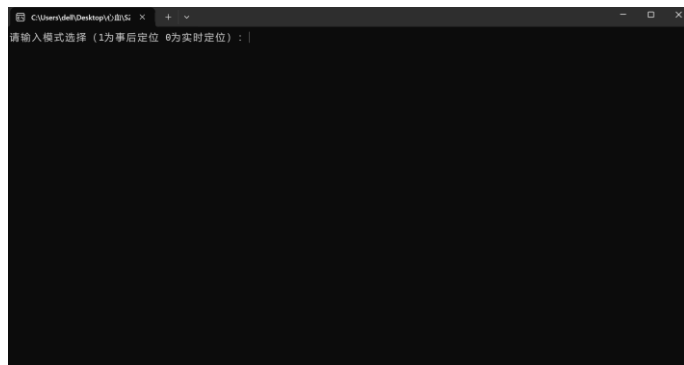
[2] 实时模式：通过串口实时接收来自 GNSS 接收机的数据流。

2. 输出:

- 1) 数据内容：软件能够计算并输出接收机的三维坐标位置、速度、钟差以及定位精度指标；
- 2) 坐标系统：所有位置解算结果均统一在 WGS-84 坐标系下；
- 3) 输出形式：定位结果以历元为单位，在 Windows 控制台窗口实时滚动显示。

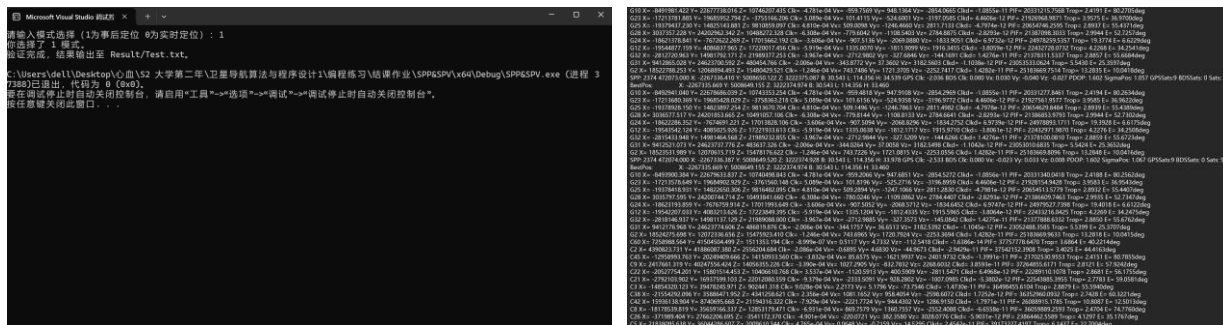
1.3 完成情况

打开 SPP&SPV.sln 文件，在配置好 Eigen 库后，点击运行按钮即显示：



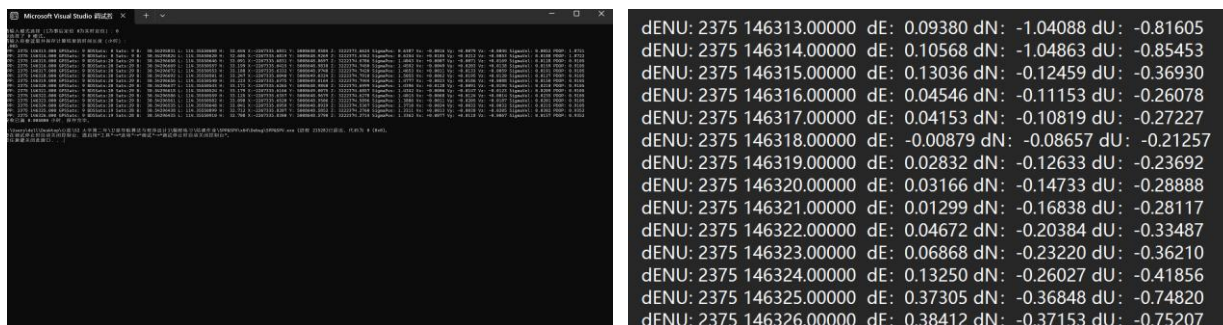
让用户选择，1 为事后定位，0 为实时定位。

选择 1:



用户选择“1”事后定位后，软件将自动读取文件夹中原有的 data/oem719-202504011900-1.bin 二进制文件，并将卫星位置与接收机位置的计算结果输出在 Result/Test.txt 文本文件中。

选择 0:



用户选择“0”实时定位后，软件将询问用户需要收集多长时间（小时）的实时数据，在用户输入完数字后，进行实时定位解算，并将接收机定位结果输出在 Windows 控制台上。并且软件会将定位误差保存在 Result/Calculation_error.txt 文本文件中，可以用于后续使用画图软件进行定位误差分析。

第二章 算法与程序设计

2.1 时间与坐标转换

2.1.1 时间转换

时间转换的整体思路：

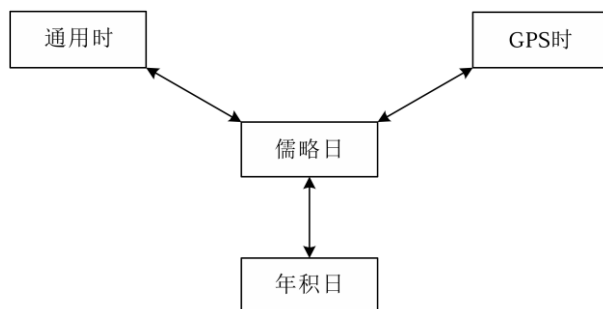


图 2-1-1 通用时、GPS 时、儒略日与年积日相互转换整体思路图

2.1.1.1 通用时转化为简化儒略日

1. 数学原理：（通用时（Y,M,D,UT）→MJD）

$$JD = INT[365.25y] + INT[30.6001(m+1)] + D + \frac{UT}{24} + 1720981.5$$

$$MJD = JD - 2400000.5$$

其中，若 $M \leq 2$ ，则 $y=Y-1$ ， $m=M+12$ ；否则 $y=Y$ ， $m=M$ 。

2. 通用时结构体定义

```

1. struct COMMONTIME {
2.     short Year;           // 年份
3.     unsigned short Month; // 月份
4.     unsigned short Day;   // 日
5.     unsigned short Hour;  // 小时
6.     unsigned short Minute; // 分钟
7.     double Second;        // 秒（可带小数）
8.
9.     COMMONTIME() : Year(0), Month(0), Day(0), Hour(0), Minute(0), Second(0.0) {}
10. };
  
```

3. C++函数实现

```

1. void CommonTimeToMjdTime(const COMMONTIME* ct, MJDTIME* mjd)
2. {
3.     double JD = int(365.25 * year) + int(30.6001 * (month + 1)) + day + second / 864
       00.0 + 1720981.5;
4.     double MJD = JD - 2400000.5;
5.     mjd->Days = int(MJD);
6.     mjd->FracDay = second / 86400.0;
7. }

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-1-1 通用时转化为简化儒略日函数功能与说明表格

函数	内容
名称	CommonTimeToMjdTime
功能	将通用时转化为简化儒略日
输入	COMMONTIME 结构体
输出	MJDTIME 结构体

2.1.1.2 简化儒略日转化为通用时

1. 数学原理：(MJD→通用时 (Y,M,D,UT))

$$\begin{aligned}
 &\text{中间变量:} \quad a = \text{INT}(JD + 0.5) \quad b = a + 1537 \\
 c = \text{INT}\left(\frac{b - 122.1}{365.25}\right) \quad &d = \text{INT}(365.25 \cdot c) \quad e = \text{INT}\left(\frac{b - d}{30.6001}\right)
 \end{aligned}$$

$$\text{日} \quad D = b - d - \text{INT}(30.6001 \cdot e) + \text{FRAC}(JD + 0.5)$$

$$\text{月} \quad M = e - 1 - 12 \cdot \text{INT}\left(\frac{e}{14}\right)$$

$$\text{年} \quad Y = c - 4715 - \text{INT}\left(\frac{7 + M}{10}\right)$$

$$\text{周} \quad N = \text{mod}(\text{INT}(JD + 0.5), 7)$$

2. 简化儒略日结构体定义

```

1. struct MJDTIME {
2.     int Days;           // 整数天数
3.     double FracDay;     // 小数天部分 (0~1)
4.     MJDTIME() : Days(0), FracDay(0.0) {}
5. };

```

3. C++函数实现

```

1. void MjdTimeToCommonTime(const MJDTIME* mjd, COMMONTIME* ct)
2. {
3.     double JD = mjd->Days + mjd->FracDay + 2400000.5;
4.     .....
5.     double seconds_of_day = mjd->FracDay * 86400.0;
6.     ct->Hour = int(seconds_of_day / 3600);
7.     ct->Minute = int((seconds_of_day - ct->Hour * 3600) / 60);
8.     ct->Second = seconds_of_day - ct->Hour * 3600 - ct->Minute * 60;
9. }

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-1-2 简化儒略日转化为通用时函数功能与说明表格

函数	内容
名称	MjdTimeToCommonTime
功能	简化儒略日转化为通用时
输入	MJDTIME 结构体
输出	COMMONTIME 结构体

2.1.1.3 GPS 时转化为简化儒略日

1. 数学原理：(GPS 时 (GPST) → MJD)

$$\text{MJD} = 44244 + \text{GPSWeek} \times 7 + \frac{\text{SecOfWeek}}{86400}$$

2. GPS 时结构体定义

```

1. struct GPSTIME
2. {
3.     unsigned short Week;
4.     double SecOfWeek;
5.
6.     GPSTIME() : Week(0), SecOfWeek(0.0) {}
7. };

```

3. C++函数实现

```

1. void GpsTimeToMjdTime(const GPSTIME* gps, MJDTIME* mjd)
2. {
3.     double MJD = 44244.0 + gps->Week * 7 + gps->SecOfWeek / 86400.0;
4.     mjd->Days = int(MJD);
5.     mjd->FracDay = (gps->SecOfWeek - int(gps->SecOfWeek / 86400.0) * 86400.0) / 86400.0;
6. }

```

该函数的功能与说明如下表：

表 2-1-3 GPS 时转化为简化儒略日函数功能与说明表格

函数	内容
名称	GpsTimeToMjdTime
功能	GPS 时转化为简化儒略日
输入	GPSTIME 结构体
输出	MJDTIME 结构体

2.1.1.4 简化儒略日转化为 GPS 时

1. 数学原理：(MJD→GPS 时 (GPST))

$$\text{GPSWeek} = \text{INT} \left[\frac{(\text{MJD} - 44244)}{7} \right]$$

$$\text{SecOfWeek} = (\text{MJD} - 44244 - \text{GPSWeek} \cdot 7) \cdot 86400$$

2. C++实现

```

1. void MjdTimeToGpsTime(const MJDTIME* mjd, GPSTIME* gps)
2. {
3.     gps->Week = int((mjd->Days - 44244.0) / 7.0 + mjd->FracDay / 7.0);

```

```
4.     gps->SecOfWeek = (mjd->Days + mjd->FracDay - 44244.0 - gps->Week * 7.0) * 86400.0;  
5. }
```

该函数的功能与说明如下表：

表 2-1-4 简化儒略日转化为 GPS 函数功能与说明表格

函数	内容
名称	MjdTimeToGpsTime
功能	简化儒略日转化为 GPS 时
输入	MJDTIME 结构体
输出	GPSTIME 结构体

2.1.1.5 通用时与 GPS 时相互转换

1. 数学原理：（借助 MJD 中间变量）

通用时 (Y,M,D,UT) ⇔ MJD ⇔ GPS 时 (GPST)

2. C++实现

```
1. void GpsTimeToCommonTime(const GPSTIME* gps, COMMONTIME* ct)  
2. {  
3.     MJDTIME mjd1;  
4.     GpsTimeToMjdTime(gps, &mjd1);  
5.     MjdTimeToCommonTime(&mjd1, ct);  
6. }
```

```
1. void CommonTimeToGpsTime(const COMMONTIME* ct, GPSTIME* gps)  
2. {  
3.     MJDTIME mjd1;  
4.     CommonTimeToMjdTime(ct, &mjd1);  
5.     MjdTimeToGpsTime(&mjd1, gps);  
6. }
```

这两个函数的功能与说明如下表：

表 2-1-5 通用时与 GPS 时相互转化函数功能与说明表格

函数	内容
名称	GpsTimeToCommonTime、CommonTimeToGpsTime
功能	通用时转化为 GPS 时、GPS 时转化为通用时
输入	COMMONTIME 结构体、GPSTIME 结构体
输出	GPSTIME 结构体、COMMONTIME 结构体

2.1.2 坐标转换

坐标转换的整体思路：

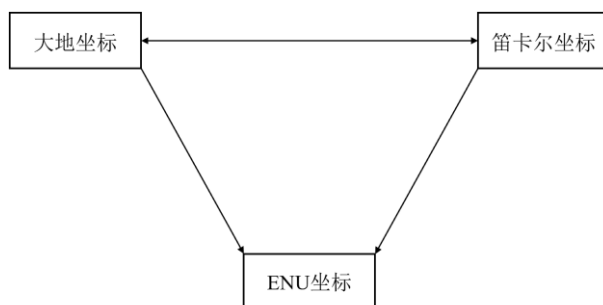


图 2-1-2 大地坐标、笛卡尔坐标与 ENU 坐标相互转换整体思路图

2.1.2.1 大地坐标转换为笛卡尔坐标

1. 数学原理：(BLH→XYZ)

$$\begin{aligned}
 X &= (N + H) \cdot \cos B \cdot \cos L \\
 Y &= (N + H) \cdot \cos B \cdot \sin L \\
 Z &= [N \cdot (1 - e^2) + H] \cdot \sin B
 \end{aligned}$$

其中， $N = \frac{a}{\sqrt{1 - e^2 \sin^2 B}}$ 为卯酉圈曲率半径， a 为地球长半轴， e^2 为第一偏心率平方。

2. 大地坐标结构体定义

```

1. union GEOCOORD
2. {
3.     struct {
4.         double latitude;    // 纬度
5.         double longitude;   // 经度
6.         double height;      // 高程
7.     };
8.     double Blh[3];
9. };
  
```

3. C++实现

```

1. void BLHToXYZ(const double BLH[3], double XYZ[3], const double R, const double F)
2. {
3.     double a = R;                // 长半轴
  
```



```
4.      .....
5.      XYZ[0] = (N + H) * cosB * cosL;
6.      XYZ[1] = (N + H) * cosB * sinL;
7.      XYZ[2] = (N * (1.0 - e2) + H) * sinB;
8. }
```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-1-6 大地坐标转化为笛卡尔坐标函数功能与说明表格

函数	内容
名称	BLHToXYZ
功能	大地坐标转化为笛卡尔坐标
输入	GEOCOOR 结构体
输出	XYZ 结构体

2.1.2.2 笛卡尔坐标转换为大地坐标

1. 数学原理：(XYZ→BLH)

$$B=\arctan\left(\frac{Z+\Delta Z}{\sqrt{X^2+Y^2}}\right)$$
$$L=\arctan\left(\frac{Y}{X}\right)$$
$$H=\sqrt{X^2+Y^2+(Z+\Delta Z)^2}-N$$

其中 $\Delta Z = Ne^2 \sin B$ 。

2. 笛卡尔坐标结构体定义

```
1. union XYZ
2. {
3.     struct {
4.         double x;
5.         double y;
6.         double z;
7.     };
8.     double xyz[3];
9. };
```

3. C++实现

```

1. void XYZToBLH(const double XYZ[3], double BLH[3], const double R, const double F)
2. {
3.     double a = R;
4.     .....
5.     BLH[0] = B * Deg;           // 纬度 (度)
6.     BLH[1] = atan2(y, x) * Deg; // 经度 (度)
7.     BLH[2] = H;                 // 高程 (米)
8. }

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-1-7 笛卡尔坐标转化为大地坐标函数功能与说明表格

函数	内容
名称	XYZToBLH
功能	笛卡尔坐标转化为大地坐标
输入	XYZ 结构体
输出	GEOCOORD 结构体

2.1.2.3 测站地平坐标定位误差计算

1. 数学原理：(((BLH_r→XYZ) + (X₀,Y₀,Z₀)) →dE,dN,dH)

$$\begin{bmatrix} dE \\ dN \\ dH \end{bmatrix} = R \begin{bmatrix} x - x_0 \\ y - y_0 \\ z - z_0 \end{bmatrix} = \begin{bmatrix} -\sin L & \cos L & 0 \\ -\sin B \cos L & -\sin B \sin L & \cos B \\ \cos B \cos L & \cos B \sin L & \sin B \end{bmatrix} \begin{bmatrix} x - x_0 \\ y - y_0 \\ z - z_0 \end{bmatrix}$$

其中，xyz 是测站的 BLH 转化为笛卡尔坐标系的结果；x₀,y₀,z₀ 是接收机精确坐标。

2. 测站地平坐标结构体定义

```

1. union NEU
2. {
3.     struct {
4.         double dE;
5.         double dN;
6.         double dU;
7.     };
8.     double Neu[3]; // ENU
9. };

```

3. C++实现

```

1. void BlhToNeuMat(const GEOCOOR* Blh, Matrix3d& Mat)
   at)
2. {
3.     Mat << -sinL, cosL, 0.0,
4.           -sinB * cosL, -sinB * sinL, cosB,
5.           cosB * cosL, cosB * sinL, sinB;
6. }

```

```

1. void CompEnudPos(const double Xs[], const double
   Xr[], const GEOCOOR* Blh, double dENU[])
2. {

```

```

3.     Vector3d Delta;
4.     Delta << Xs[0] - Xr[0], Xs[1] - Xr[1], Xs[2]
   - Xr[2];
5.
6.     Matrix3d R;
7.     BlhToNeuMat(Blh, R); // E/N/U 方向旋转矩阵
8.
9.     Vector3d d_neu = R * Delta;
10.
11.    dENU[0] = d_neu(0); // E
12.    dENU[1] = d_neu(1); // N
13.    dENU[2] = d_neu(2); // U
14. }

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-1-8 测站地平坐标定位误差计算函数说明表格

函数	内容 1	内容 2
名称	BlhToNeuMat	CompEnudPos
功能	生成 Neu 旋转矩阵	计算测站地平坐标定位误差
输入	GEOCOOR 结构体	Xs[] Xr[]数组，GEOCOOR 结构体
输出	Matrix3d 矩阵	NEO 结构体结构体

2.1.2.4 卫星的高度角与方位角计算

1. 数学原理：(((BLHr→XYZ) + (X₀,Y₀,Z₀)) →Elev,Azim)

$$\begin{bmatrix} dE \\ dN \\ dH \end{bmatrix} = R \begin{bmatrix} x^s - x_0 \\ y^s - y_0 \\ z^s - z_0 \end{bmatrix} = \begin{bmatrix} -\sin L & \cos L & 0 \\ -\sin B \cos L & -\sin B \sin L & \cos B \\ \cos B \cos L & \cos B \sin L & \sin B \end{bmatrix} \begin{bmatrix} x^s - x_0 \\ y^s - y_0 \\ z^s - z_0 \end{bmatrix}$$

$$\text{高度角 Elev} = \text{atan}\left(\frac{dH}{\sqrt{dN^2 + dE^2}}\right)$$

$$\text{方位角 Azim} = \text{atan2}(dE, dN)$$

其中，xyz 是卫星的 BLH 转化为笛卡尔坐标系的结果。

2. C++实现

```
1. void CompSatElAz(const double Xr[], const double Xs[], const GEOCOOR* Blh, double* E
   lev, double* Azim)
2. {
3.     double dENU[3] = { 0.0, 0.0, 0.0 };
4.     .....
5.     *Elev = atan2(dENU[2], EN) * Deg;
6.     *Azim = atan2(dENU[0], dENU[1]) * Deg; // E 在前、N 在后
7.     if (*Azim < 0.0) *Azim += 360.0;
8. }
```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-1-9 计算卫星高度角与方位角函数说明表格

函数	内容
名称	CompSatElAz
功能	计算卫星高度角与方位角
输入	Xr[] Xs[]数组，GEOCOOR 结构体
输出	Elev, Azim

2.2 NovATel 数据获取

2.2.1 数据格式

本项目使用的 GNSS 数据，是由 NoVATel 公司的 OEM 7 接收机生成，采用二进制格式记录，通过串口或者文件读取获得。

1. 数据帧结构：

OEM 7 的每条消息均由以下三部分组成：

表 2-2-1 OEM 7 的消息的三个重要部分表格

部分	说明
Header	固定 28 字节，包含同步字、消息类型、时间信息等
Data	可变长度，存储实际 GNSS 数据，如伪距、星历等
CRC	固定 4 字节，32 位 CRC 校验码，用于数据完整性验证

其中 Header 部分前三字节为同步标志：0xAA 0x44 0x12，用于识别一帧数据的起始。

2. 主要消息：

常见的 Message ID 类型有：

表 2-2-2 常见的 Message ID 类型以及内容表格

Message ID	内容
43	观测值
7	GPS 星历
1696	BDS 星历
42	位置解

3. CRC 校验说明：

每帧数据尾部的 4 字节 CRC 用于校验 Header 与 Data 的内容是否正确，确保数据传输可靠。CRC 多项式为 0xED888320，本项目通过代码实现校验。

2.2.2 解码函数

2.2.2.1 CRC 校验

以下是 CRC 校验代码展示：

```

1. unsigned int crc32(const unsigned char* buff, int len)
2. {
3.     int i, j;
4.     unsigned int crc = 0;
5.     for (i = 0; i < len; i++)
6.     {
7.         crc ^= buff[i];
8.         for (j = 0; j < 8; j++)
9.         {
10.            if (crc & 1)
11.                crc = (crc >> 1) ^ POLYCRC32;
12.            else
13.                crc >>= 1;
14.        }
15.    }
16.    return crc;
17.}

```

2.2.2.2 解码主函数

解码主函数大致代码如下：

```

1. int DecodeNovOem7Dat(unsigned char buff[], int&
   len, EPOCHOBS* obs, GPSEPHREC geph[], GPSEPHREC
   beph[], POSRES* pos)
2. {
3.     while (i < len)
4.     {
5.         // 1、查找同步字符 AA 44 12
6.         .....
7.         // 2、检查是否有足够的字节读取完整的消息头
           (28 字节)
8.         .....
9.         // 3、检查整条消息是否完整 (头 28 字
           节 + MsgLen 字节 + CRC 4 字节)
10.        .....
11.        // 4、CRC 检验
12.        .....

```

```

13.        // 5、根据消息 ID 解码
14.        switch (MsgID)
15.        {
16.            case 43: // 观测数据消息
17.            case 7: // GPS 星历
18.            case 1696: // BDS 星历
19.            case 42: // 位置解
20.            default:
21.                // 未识别的消息，跳过
22.            }
23.        // 5 (继续)：跳过整条消息
24.        .....
25.    }
26.    // 6、循环结束，处理剩余数据
27.    .....
28.}

```

本项目整体解码流程为：首先查找同步头 AA 44 12；然后解出 Header 字段中的时间、ID、长度等；然后根据 ID 选择对应的解码函数；最后进行 CRC 校验，若通过则保存数据。

2.2.3 字节提取

本处举一个具体的字节提取函数的代码示例：

```
1. double R8(unsigned char* p) // 提取 8 字节 double
2. {
3.     double r;
4.     memcpy(&r, p, 8);
5.     return r;
6. }
```

以下是所有字节提取函数的总列表格：

表 2-2-3 字节提取函数总列表格

名称	功能
R8	提取 8 字节 double
R4	提取 4 字节 float
I4	提取 4 字节有符号整数
UI4	提取 4 字节无符号整数
I2	提取 2 字节有符号整数
UI2	提取 2 字节无符号整数

2.2.4 观测数据提取

2.2.4.1 观测数据提取

1. 每个历元观测数据结构体定义：

```
1. struct EPOCHOBS
2. {
3.     GPSTIME    Time;           // 当前历元的时间 GPS 时间，含周和周内秒
4.     short      SatNum;         // 当前历元观测到的卫星数量
5.     SATOBSDATA SatObs[MAXCHANUM]; // 每颗卫星的观测数据（如伪距、载波等）
6.     SATPVT     SatPVT[MAXCHANUM]; // 每颗卫星的位置、速度、钟差计算结果
```

```

7.      MWGF      ComObs[MAXCHANNUM];           // 每颗卫星的组合观测量MW 组合、GF 组合
8.      double    Pos[3];                       // 当前历元接收机的位置 (XYZ 坐标)
9.
10.     EPOCHOBS()
11.     {
12.         SatNum = 0;
13.         Pos[0] = Pos[1] = Pos[2] = 0.0;
14.     }
15. };

```

2. C++实现:

```

1.  void DecodeRange(unsigned char* data, EPOCHOBS*
    obs)
2.  {
3.      .....
4.      //1、从消息头中解码得到观测时刻，该时刻为接收机钟
        表面时，用GPSTIME 结构体表示。
5.      .....
6.      //2、解码得到观测值数量，为所有卫星所有信号观测值
        的总数。
7.      .....
8.      //3、对所有信号观测值进行循环解码
9.      for (i = 0, p += 4; i < ObsNum; i++, p += 44
        ) // p+4 是因为要跳过ObsNum4 个字节，p+44 是因为一
            个观测数据固定长度44 个字节
10.     {
11.         //⑤ 解码得到跟踪状态标记，从中取出
            Phase Lock flag/Code Locked / flag / Parity know
            n flag / Satellite system / signal type 等数据
12.         .....
13.         //⑥ 如果卫星系统不是GPS 或BDS，continue 至③
14.         //⑦ 如果GPS 卫星的信号类型不是L1 C/A 或者
            L2PY, BDS 卫星不是B1I 或B3I，continue 至③，并记录信
            号频率类型，第一频率s=0，第二频率s=1;
15.         .....
16.         //⑧ 解码得到卫星号Prn 以及卫星系统号，在当前
            观测值结构体中进行搜索，如果找到相同的卫星，将解码的
            观测值填充到该卫星对应的数组中；如果在当前已解码的卫
            星数据中没有发现，则填充到现有数据的末尾。
17.         .....
18.     }
19.     //4. 统计有效的卫星数量
20.     .....
21. }

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-2-4 观测数据提取函数的说明表格

函数	内容
名称	DecodeRange
功能	提取观测数据
输入	二进制数据 (AscII)
输出	EPOCHOBS 结构体

2.2.4.2 GPS 广播星历提取

1. GPS 卫星星历结构体定义:

```

1.  struct GPSEPHREC {
2.      // === 卫星基本信息 ===
3.      short PRN;           // 卫星编号
4.      GNSSSys System;      // 所属系统 (GPS、BDS
                           // 等)
5.
6.      // === 时间参数 ===
7.      GPSTIME TOC;         // 钟差参考时间
8.      GPSTIME TOE;         // 轨道参考时间
9.
10.     // === 卫星钟差参数 ===
11.     double ClkBias;       // 偏差 [s]
12.     double ClkDrift;      // 漂移 [s/s]
13.     double ClkDriftRate;  // 漂移率 [s/s2]
14.
15.     // === 轨道基本参数 ===
16.     double SqrtA;         // 轨道长半轴的平方
                           // 根 [√m]
17.     double e;            // 偏心率
18.     double M0;           // 平近点角 [rad]
19.     double i0;           // 倾角 [rad]
20.     double omega;        // 近地点幅角 [rad]
21.     double OMEGA;        // 升交点赤经 [rad]
22.
23.     // === 轨道改正参数 ===
24.     double Crs, Crc;      // 径向改正项 [m]
25.     double Cus, Cuc;      // 幅角改正项 [rad]
26.     double Cis, Cic;      // 升交点改正项 [rad]
27.     double DeltaN;        // 平均角速度改
                           // 正 [rad/s]
28.     double OMEGADot;      // 升交点赤经变化
                           // 率 [rad/s]
29.     double iDot;          // 倾角变化率 [rad/s]
30.
31.     // === 其他参数 ===
32.     int SVHealth;         // 卫星健康标志 (0 为健
                           // 康)
33.     double IODE, IODC;    // 星历编号
34.     double TGD1, TGD2;    // 群延迟参数 [s]
35.
36.     // 构造函数: 全部初始化为 0 或默认
37.     GPSEPHREC() {
38.         PRN = 0; System = UNKS;
39.         TOC = TOE = GPSTIME();
40.         ClkBias = ClkDrift = ClkDriftRate = 0.0;
41.         SqrtA = e = M0 = i0 = omega = OMEGA = 0.0;
42.         Crs = Crc = Cus = Cuc = Cis = Cic = 0.0;
43.         DeltaN = OMEGADot = iDot = 0.0;
44.         IODE = IODC = TGD1 = TGD2 = 0.0;
45.         SVHealth = 0;
46.     }
47. };

```

2. C++实现

```

1. void DecodeGpsEphem(unsigned char* data, GPSEPHREC geph[])
2. {
3.     //=== 解析 PRN & 检查 =====
4.     .....
5.     //=== 时间 =====
6.     .....
7.     //=== 星历标识 =====
8.     .....
9.     //=== 卫星健康 =====
10.    .....
11.    //=== 轨道六根数 & 改正项 =====
12.    .....
13.    //=== 钟差参数 =====
14.    .....

```

```

15.    //=== 组延迟 (TGD) =====
16.    .....
17.    //=== Anti-spoofing、URA、N 暂不处理 =====
18.}

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-2- 5 GPS 卫星星历提取函数的说明表格

函数	内容
名称	DecodeGpsEphem
功能	提取 GPS 卫星星历
输入	二进制数据 (Ascii)
输出	GPSEPHREC 结构体

2.2.4.3 BDS 广播星历提取

BDS 广播星历提取过程与 GPS 基本相同，使用的结构体是 GPSEPHREC，这里不再对结构体与 C++实现进行说明，以下是该函数的功能与说明表格：

表 2-2- 6 BDS 卫星星历提取函数的说明表格

函数	内容
名称	DecodeBdsEphem
功能	提取 BDS 卫星星历
输入	二进制数据 (Ascii)
输出	GPSEPHREC 结构体

2.2.4.4 定位结果提取

1. 定位结果结构体定义：

```

1. struct POSRES {
2.     GPSTIME Time;           // 定位时间
3.     double Pos[3];          // 接收机位置：经度、纬度、高程（单位：度、度、米）
4.
5.     double SigmaPos;        // 三维位置精度（标准差）
6.     double PDOP;            // PDOP 精度因子

```

```

7.
8.     unsigned char SatNum_tracked; // 跟踪卫星数
9.     unsigned char SatNum_used;    // 解算使用的卫星数
10.
11.    unsigned int sol_status;       // 解状态
12.    unsigned int pos_type;        // 位置类型
13.
14.    POSRES() {
15.        Pos[0] = Pos[1] = Pos[2] = 0.0;
16.        SigmaPos = PDOP = 0.0;
17.        SatNum_tracked = SatNum_used = 0;
18.        sol_status = pos_type = 0;
19.    }
20.};

```

2. C++实现:

```

1. void Decode_psrpos(unsigned char* data, POSRES* pos)
2. {
3.     //=== 1、解状态 & 位置类型 =====
4.     .....
5.     //=== 2、位置 =====
6.     .....
7.     //=== 3、大地水准面高度差 =====
8.     .....
9.     //=== 4、标准差 =====
10.    .....
11.    // 组合成三维位置标准差（平方和开根号）
12.    .....
13.    //=== 5、卫星数量 =====
14.    .....
15.    //=== 6、未提供字段 =====
16.    .....
17.    //=== 7、时间 =====
18.    .....
19.}

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-2-7 定位结果提取的函数说明表格

函数	内容
名称	Decode_psrpos
功能	提取定位结果

输入

二进制数据（Ascii）

输出

POSRES 结构体

以下是观测数据、卫星星历、位置解算的数据位置说明表格：

Field	Field type	Description	Format	Binary bytes	Binary offset
1	Log header	RAWEPHEM header For information about log headers, see ASCII , Abbreviated ASCII or Binary .		H	0
2	PRN	Satellite PRN number	Ulong	4	H
3	ref week	Ephemeris reference week number	Ulong	4	H+4
4	ref secs	Ephemeris reference time (s)	Ulong	4	H+8
5	subframe1	Subframe 1 data	Hex[30]	30	H+12
6	subframe2	Subframe 2 data	Hex[30]	30	H+42
7	subframe3	Subframe 3 data	Hex[30]	30	H+72
8	xxxx	32-bit CRC (ASCII and Binary only)	Hex	4	H+102
9	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

Field	Field type	Description	Format	Binary bytes	Binary offset
1	Log header	BDOEPHEMERIS header For information about log headers, see ASCII , Abbreviated ASCII or Binary .		H	0
2	satellite ID	ID/ranging code	Ulong	4	H
3	Week	Week number in BeiDou time (BDT) since the BeiDou system time start epoch (January 1, 2006)	Ulong	4	H+4
4	URA	User range accuracy (m). This is the evaluated URA/URA lookup-table value.	Double	8	H+8
5	health 1	Autonomous satellite health flag. 0 means broadcasting satellite is good and 1 means not.	Ulong	4	H+16
6	tgdt1	Equipment group delay differential for the B1 signal (s)	Double	8	H+20
7	tgdt2	Equipment group delay differential for the B2 signal (s)	Double	8	H+28
8	AODC	Age of data, clock	Ulong	4	H+36
9	toc	Reference time of clock parameters (s)	Ulong	4	H+40
10	a_0	Constant term of clock correction polynomial (s)	Double	8	H+44
11	a_1	Linear term of clock correction polynomial (s/s)	Double	8	H+52
12	a_2	Quadratic term of clock correction polynomial (s/s ²)	Double	8	H+60
13	AODE	Age of data, ephemeris	Ulong	4	H+68
14	toe	Reference time of ephemeris parameters (s)	Ulong	4	H+72
15	RootA	Square root of semi-major axis (sqrt(m))	Double	8	H+76
16	ecc	Eccentricity (dimensionless)	Double	8	H+84
17	ω	Argument of perigee (radians)	Double	8	H+92
18	ΔN	Mean motion difference from computed value (radians/s)	Double	8	H+100
19	M_0	Mean anomaly at reference time (radians)	Double	8	H+108
20	Ω_0	Longitude of ascending node of orbital plane computed according to reference time (radians)	Double	8	H+116
21	$\dot{\Omega}$	Rate of right ascension (radians/s)	Double	8	H+124
22	i_0	Inclination angle at reference time (radians)	Double	8	H+132
23	IDOT	Rate of inclination angle (radians/second)	Double	8	H+140
24	c_{uc}	Amplitude of cosine harmonic correction term to the argument of latitude (radians)	Double	8	H+148
25	c_{us}	Amplitude of sine harmonic correction term to the argument of latitude (radians)	Double	8	H+156
26	c_{rc}	Amplitude of cosine harmonic correction term to the orbit radius (m)	Double	8	H+164
27	c_{rs}	Amplitude of sine harmonic correction term to the orbit radius (m)	Double	8	H+172
28	c_{ic}	Amplitude of cosine harmonic correction term to the angle of inclination (radians)	Double	8	H+180
29	c_{is}	Amplitude of sine harmonic correction term to the angle of inclination (radians)	Double	8	H+188
30	xxxx	32-bit CRC (ASCII and Binary only)	Ulong	4	H+196
31	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

Field	Field type	Description	Format	Binary bytes	Binary offset
1	Log header	GPS/EPHEMERIS header For information about log headers, see ASCII , Abbreviated ASCII or Binary .		H	0
2	PRN	Satellite PRN number	Ulong	4	H
3	tow	Time stamp of subframe 1 (s)	Double	8	H+4
4	health	Health status • bits 5-8: 5-bit health code as defined in IS-GPS-200. • bit 9: start flag	Ulong	4	H+12
5	ODE1	Issue of ephemeris data 1	Ulong	4	H+16
6	ODE2	Issue of ephemeris data 2	Ulong	4	H+20
7	week	See week number computed from 2 count week (GPS reference week number)	Ulong	4	H+24
8	$\pm$ week	2 count week number. This is the week number from subframe 1 of the ephemeris. The 'see week' (field #7) is derived from this to account for rollover	Ulong	4	H+28
9	toc	Reference time for ephemeris (s)	Double	8	H+32
10	A	Semi-major axis (m)	Double	8	H+40
11	ΔN	Mean motion difference (radians/s)	Double	8	H+48
12	M_0	Mean anomaly of reference time (radians)	Double	8	H+56
13	ecc	Eccentricity (dimensionless)	Double	8	H+64
14	ω	Argument of perigee (radians)	Double	8	H+72
15	cos	Amplitude of cosine harmonic correction term to the argument of latitude (radians)	Double	8	H+80
16	sos	Amplitude of sine harmonic correction term to the argument of latitude (radians)	Double	8	H+88
17	cos	Amplitude of cosine harmonic correction term to the orbit radius (m)	Double	8	H+96
18	sos	Amplitude of sine harmonic correction term to the orbit radius (m)	Double	8	H+104
19	cos	Amplitude of cosine harmonic correction term to the angle of inclination (radians)	Double	8	H+112
20	sos	Amplitude of sine harmonic correction term to the angle of inclination (radians)	Double	8	H+120
21	i_0	Inclination angle at reference time (radians)	Double	8	H+128
22	$\dot{i}$	Rate of inclination angle (radians/s)	Double	8	H+136
23	Ω_0	Longitude of ascending node of orbit plane (radians)	Double	8	H+144
24	$\dot{\Omega}$	Rate of right ascension (radians/s)	Double	8	H+152
25	iodc	Issue of data clock	Ulong	4	H+160
26	iodu	SV clock correction term (s)	Double	8	H+168
27	igdt	Estimated group delay difference (s)	Double	8	H+172
28	a_0	Clock aging parameter (s)	Double	8	H+180
29	a_1	Clock aging parameter (s/s)	Double	8	H+188
30	a_2	Clock aging parameter (s/s ²)	Double	8	H+196
31	AS	Axis spoofing on 0 = FALSE 1 = TRUE	Bool	4	H+204
32	N	Corrected mean motion (radians/s)  This field is computed by the receiver	Double	8	H+208
33	URA	User Range Accuracy variance (m) ² The URA specified in the URA index transmitted in the ephemerides can be converted to a normal standard deviation value using an algorithm listed here. We publish the square of the normal value (variance). The correspondence between the original URA index and the value output is shown in Table 1 (GPS variance)	Double	8	H+216
34	xxxx	32-bit CRC (ASCII and Binary only)	Ulong	4	H+224
35	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

Field	Field type	Description	Format	Binary bytes	Binary offset
1	Log header	BESTPOS header For information about log headers, see ASCII , Abbreviated ASCII or Binary .		H	0
2	sol stat	Solution status, see Table: Solution status	Enum	4	H
3	pos type	Position type, see Table: Position or velocity type	Enum	4	H+4
4	lat	Latitude (degrees)	Double	8	H+8
5	lon	Longitude (degrees)	Double	8	H+16
6	hgt	Height above mean sea level (metres)	Double	8	H+24
7	undulation	Undulation - the relationship between the geoid and the ellipsoid (m) of the chosen datum  When using a datum other than WGS84, the undulation value also includes the vertical shift due to differences between the datum in use and WGS84.	Float	4	H+32
8	datum id#	Datum ID number 61 = WGS84 63 = USER	Enum	4	H+36
9	lat σ	Latitude standard deviation (m)	Float	4	H+40
10	lon σ	Longitude standard deviation (m)	Float	4	H+44
11	hgt σ	Height standard deviation (m)	Float	4	H+48
12	stn id	Base station ID	Char[4]	4	H+52
13	diff_age	Differential age in seconds	Float	4	H+56
14	sol_age	Solution age in seconds	Float	4	H+60
15	#SVs	Number of satellites tracked	Uchar	1	H+64
16	#solnSVs	Number of satellites used in solution	Uchar	1	H+65
17	#solnL1SVs	Number of satellites with L1/E1B1 signals used in solution	Uchar	1	H+66
18	#solnMultSVs	Number of satellites with multi-frequency signals used in solution	Uchar	1	H+67
19	Reserved		Hex	1	H+68
20	ext sol stat	Extended solution status (see Table: Extended solution status)	Hex	1	H+69
21	Galileo and BeiDou sig mask	Galileo and BeiDou signals used mask (see Table: Galileo and BeiDou signal-used mask)	Hex	1	H+70
22	GPS and GLONASS sig mask	GPS and GLONASS signals used mask (see Table: GPS and GLONASS signal-used mask)	Hex	1	H+71
23	xxxx	32-bit CRC (ASCII and Binary only)	Hex	4	H+72
24	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

图 2-2- 1 观测数据格式（上）卫星星历（中）定位结果（下）

访问网站：

[RAWEPHEM](#)

[GPSEPHHEM](#)

[BDSEPHHEMERIS](#)

[BESTPOS](#)

2.3 卫星位置计算

2.3.1 GPS 卫星

2.3.1.1 卫星位置与速度

1. 数学原理：

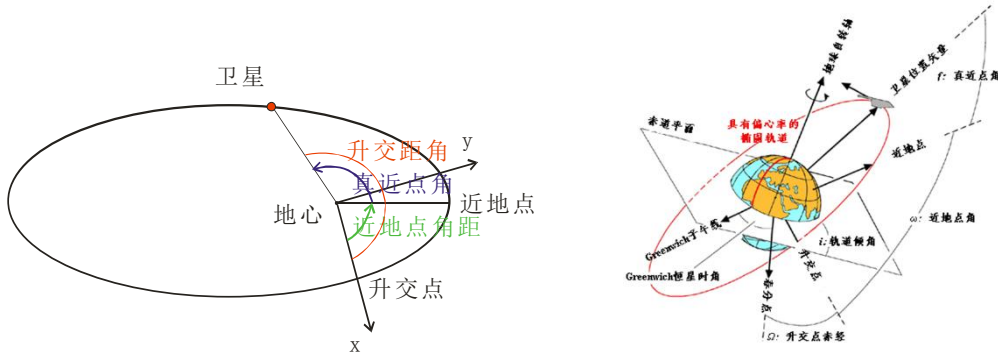


图 2-3-1 轨道平面坐标系（左）轨道参数（右）

1) 步骤一 计算参考时刻卫星运行的平均角速度

$$n_0 = \frac{\sqrt{GM}}{(\sqrt{A})^3}$$

其中 GM 为地球引力常数，使用 GPS 系统时在 WGS84 系中定义为 $3.986005 \times 10^{14} \text{ m}^3/\text{s}^2$ 。考虑摄动参数时：

$$n = n_0 + \Delta n$$

其中 n 表示卫星运行的平均角速度。

2) 步骤二 计算观测瞬间卫星的平近点角

$$M_k = M_0 + n \cdot (t - t_{oe})$$

其中计算 $t - t_{oe}$ 时使用的是观测时间的 GPS 周内秒与星历参考时刻，因此需要考虑跨周的影响：当 $t - t_{oe} < -302400$ 时，则 $t - t_{oe} + 604800$ ；当 $t - t_{oe} > 302400$ 时，则 $t - t_{oe} - 604800$ 。

3) 步骤三 计算偏近点角

$$E(t) = M(t) + e \cdot \sin E(t)$$

迭代计算，首先初始化：

$$E = M$$

然后迭代更新：

$$\delta = \frac{E - e \cdot \sin E - M}{1 - e \cdot \cos E}$$

$$E = E - \delta$$

迭代终止条件为：

$$|\delta| > \varepsilon$$

其中 ε 是收敛容差，本项目中定义为 10^{-12} 。

4) 步骤四 计算真近点角

$$v_k = \arctan \left(\frac{\sqrt{1-e^2} \sin E_k}{\cos E_k - e} \right)$$

5) 步骤五 计算升交角距

$$\Phi_k = v_k + w$$

6) 步骤六 计算摄动改正

$$\begin{aligned} \delta u_k &= C_{uc} \cdot \cos(2\Phi_k) + C_{us} \cdot \sin(2\Phi_k) \\ \delta r_k &= C_{rc} \cdot \cos(2\Phi_k) + C_{rs} \cdot \sin(2\Phi_k) \\ \delta i_k &= C_{ic} \cdot \cos(2\Phi_k) + C_{is} \cdot \sin(2\Phi_k) \end{aligned}$$

其中 $\delta u_k, \delta r_k, \delta i_k$ 分别是升交角距、向径、轨道倾角的改正数。

7) 步骤七 进行摄动改正

$$\begin{aligned} u_k &= \Phi_k + \delta u_k \\ r_k &= r'(t) + \delta r_k \\ i_k &= i_0 + \dot{i} \cdot (t - t_{oe}) + \delta i_k \\ \Omega_k &= \Omega_0 + (\dot{\Omega} - \dot{\Omega}_e) \cdot t_k - \dot{\Omega}_e \cdot t_{oe} \end{aligned}$$

其中 u_k, r_k, i_k 分别是改正后的升交角距、向径、轨道倾角、升交点经度（地固系下）。

8) 步骤八 计算卫星在轨道平面上的位置

$$\begin{aligned} x'_k &= r_k \cdot \cos u_k \\ y'_k &= r_k \cdot \sin u_k \end{aligned}$$

9) 步骤九 计算卫星在地固坐标系的位置

$$\begin{aligned}x_k &= x'_k \cos \Omega_k - y'_k \cos i_k \sin \Omega_k \\y_k &= x'_k \sin \Omega_k + y'_k \cos i_k \cos \Omega_k \\z_k &= y'_k \sin i_k\end{aligned}$$

此处的卫星位置是未经地球自传改正的，因为信号发射时刻未知。

10) 步骤十 计算卫星在地固坐标系的速度

$$\begin{bmatrix} \dot{x}_k \\ \dot{y}_k \\ \dot{z}_k \end{bmatrix} = \begin{bmatrix} \cos \Omega_k & -\sin \Omega_k \cos i_k & -(x'_k \sin \Omega_k + y'_k \cos \Omega_k \cos i_k) & y'_k \sin \Omega_k \sin i_k \\ \sin \Omega_k & \cos \Omega_k \cos i_k & (x'_k \cos \Omega_k - y'_k \sin \Omega_k \cos i_k) & -y'_k \cos \Omega_k \sin i_k \\ 0 & \sin i_k & 0 & y'_k \cos i_k \end{bmatrix} \begin{bmatrix} \dot{r}_k \cos u_k - r'_k \dot{u}_k \sin u_k \\ \dot{r}_k \sin u_k + r'_k \dot{u}_k \cos u_k \\ \dot{\Omega} \\ \dot{i} + 2(C_{is} \cos 2\Phi_k - C_{ic} \sin 2\Phi_k) \dot{\Phi}_k \end{bmatrix}$$

此处的卫星速度是未经地球自传改正的，同样因为信号发射时刻未知。

2.3.1.2 卫星钟差与钟速

1. 数学原理:

$$\begin{aligned}\Delta t_{sv} &= ClkBias + ClkDrift \cdot (t - t_{oe}) + ClkDriftRate \cdot (t - t_{oe})^2 - \frac{2\sqrt{\mu}}{c^2} \cdot e \cdot \sqrt{A} \cdot \sin E_k \\ \frac{d(\Delta t_{sv})}{dt} &= ClkDrift + 2ClkDriftRate \cdot (t - t_{oe}) - \frac{2\sqrt{\mu}}{c^2} \cdot e \cdot \sqrt{A} \cdot \cos E_k \cdot \dot{E}_k\end{aligned}$$

其中 Δt_{sv} , $\frac{d(\Delta t_{sv})}{dt}$ 分别是卫星钟差与钟速； μ 是地球自转角速度，在这里为 WGS-84 系下。

2. 位置速度解算的结构体定义:

```
1. struct SATPVT
2. {
3.     double SatPos[3]; // 卫星在 ECEF 坐标系下的
        三维位置 (X, Y, Z), 单位: 米
4.     double SatVel[3]; // 卫星在 ECEF 坐标系下的
        三维速度 (VX, VY, VZ), 单位: 米/秒
5.     double SatClkOfst; // 卫星钟差, 单
        位: 秒 (包含卫星钟偏移、漂移及相对论效应)
6.     double SatClkSft; // 卫星钟速差, 单位: 无
        量纲 (用于精密定位中的钟速改正)
7.     double Elevation; // 卫星仰角, 单位: 弧
        度 (范围: 0 ~ π/2)
```

```
8.     double Azimuth; // 卫星方位角, 单位: 弧
        度 (范围: 0 ~ 2π, 从北方向顺时针计量)
9.     double TropCorr; // 对流层延迟改正量, 单
        位: 米 (用于消除大气延迟对信号的影响)
10.    double Tgd1, Tgd2; // 群延迟改正参数, 单
        位: 秒 (Tgd1: L1/L2 频率间偏差, Tgd2: 其他频率间偏
        差)
11.    bool Valid; // 数据有效性标志:
12.                // false=没有可用星历或
        星历过期/不健康,
13.                // true=卫星位置、钟差计
        算成功且可用
```



```

14.
15.     SATPVT()
16.     {
17.         // 初始化位置和速度为0
18.         SatPos[0] = SatPos[1] = SatPos[2] = 0.0;
19.         SatVel[0] = SatVel[1] = SatVel[2] = 0.0;
20.
21.         // 初始仰角设为90度(天顶方向), 表示默认无遮挡
22.         Elevation = PAI / 2.0;

```

```

23.
24.         // 初始化钟差、钟速差和改正参数为0
25.         SatClkOft = SatClkSft = 0.0;
26.         Tgd1 = Tgd2 = TropCorr = 0.0;
27.
28.         Azimuth = 0.0;
29.         // 默认标记为无效数据
30.         Valid = false;
31.     }
32. };

```

3. C++实现:

```

1. int CompGPSSatPVT(const int Prn, const GPSTIME*
   t, const GPSEPHREC* Eph, SATPVT* Mid)
2. {
3.     // 星历是否过期或健康判断
4.     .....
5.
6.     /* 一、计算卫星位置 */
7.     // 1、计算轨道长半轴
8.     // 2、计算平均角速度
9.     // 3、计算相对于星历参考历元的时间
10.    // 4、对平均运动角速度进行改正
11.    // 5、计算平近点角
12.    // 6、计算偏近点角
13.    // 7、计算真近点角
14.    // 8、计算升交角距
15.    // 9、计算二阶调和改正数
16.    // 10-12、 计算改正后的升交角距、轨道半径和轨道
    倾角
17.    // 13、计算卫星在轨道上面的位置
18.    // 14、计算改正后的升交点经度
19.    // 15、计算在地固坐标系下的位置
20.
21.    /* 二、计算卫星速度 */

```

```

22.     .....
23.
24.     /* 三、计算卫星钟差与钟速 */
25.     // 计算卫星钟差
26.     .....
27.     // 计算卫星钟速
28.     .....
29.     /* 四、保存结果 */
30.     Mid->SatPos[0] = X;
31.     Mid->SatPos[1] = Y;
32.     Mid->SatPos[2] = Z;
33.
34.     Mid->SatVel[0] = Vx;
35.     Mid->SatVel[1] = Vy;
36.     Mid->SatVel[2] = Vz;
37.
38.     Mid->SatClkOft = T_sv;
39.     Mid->SatClkSft = delta_T_sv;
40.
41.     Mid->Tgd1 = Eph->TGD1;
42.     Mid->Tgd2 = Eph->TGD2;
43. }

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-3-1 GPS 卫星位置速度、钟差钟速的计算函数说明表格

函数	内容
名称	CompGPSSatPVT
功能	解算 GPS 卫星位置、速度，钟差、钟速

输入	卫星号、GPSTIME GPSEPHREC 结构体
输出	SATPVT 结构体

2.3.2 BDS 卫星

2.3.2.1 卫星位置与速度

1. 数学原理：

- IGSO 和 MEO 同 GPS。
- GEO 卫星：（仅从步骤七开始往后不一致）

1) 步骤七 进行摄动改正

升交点经度摄动改正与变化率的计算的公式不同（惯性系下）：

$$\begin{aligned}\Omega_k &= \Omega_0 + \dot{\Omega}_{t_k} - \dot{\Omega}_e \cdot t_{oe} \\ \Omega_k &= \dot{\Omega}\end{aligned}$$

2) 步骤八 计算卫星在自定义坐标系中的坐标

$$\begin{aligned}x_{GK} &= x'_k \cos \Omega_k - y'_k \cos i_k \sin \Omega_k \\ y_{GK} &= x'_k \sin \Omega_k + y'_k \cos i_k \cos \Omega_k \\ z_{GK} &= y'_k \sin i_k\end{aligned}$$

3) 步骤九 计算卫星在 BDCS 坐标系中的坐标

$$\begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix} = R_z(\omega_e * t) R_x(-5^\circ) \begin{bmatrix} x_{GK} \\ y_{GK} \\ z_{GK} \end{bmatrix}$$

其中：

$$R_x(\varphi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{pmatrix} \quad R_z(\varphi) = \begin{pmatrix} \cos \varphi & \sin \varphi & 0 \\ -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

4) 步骤十 计算卫星在 BDCS 坐标系中的速度

$$\frac{d}{dt} \begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix} = R_z(\dot{\Omega}_e t_k) R_x(-5^\circ) \frac{d}{dt} \begin{bmatrix} x_{GK} \\ y_{GK} \\ z_{GK} \end{bmatrix} + \dot{R}_z(\dot{\Omega}_e t_k) R_x(-5^\circ) \begin{bmatrix} x_{GK} \\ y_{GK} \\ z_{GK} \end{bmatrix}$$

2.3.2.2 卫星钟差与钟速

同 GPS，但需注意地球自转角速度需使用 CGCS-2000 坐标系下定义的，以下是 BDS 卫星状态计算的函数说明表格：

表 2-3-2 BDS 卫星位置速度、钟差钟速的计算函数说明表格

函数	内容
名称	CompBDSSatPVT
功能	解算 BDS 卫星位置、速度，钟差、钟速
输入	卫星号、GPSTIME GPSEPHREC 结构体
输出	SATPVT 结构体

2.3.3 过期星历

1. 数学原理

首先计算时间差：

$$\Delta t = (t_{obs}^{week} - t_{TOC}^{week}) \cdot 604800 + (t_{obs}^{sec} - t_{TOC}^{sec})$$

对于 BDS 系统，需要做转换： $t_{BDT} = t_{GPST} - 1356 \text{周} - 14 \text{秒}$

接着进行有效性判断：

$$\text{Valid} = \begin{cases} \text{T}, & |\Delta t| \leq T_{\max} \text{ 且 } SVHealth = 0 \\ \text{F}, & \text{否则} \end{cases}$$

其中对于 GPS 系统，星历有效期 $T_{\max}$ 为 2 小时约等于 7500s；对于 BDS 系统，星历有效期为一小时约等于 3900s。

2. C++实现：

```

1. bool CompSatClkOff(const int Prn, const GNSSSys Sys, const GPSTIME* t, GPSEPHREC* GPSEph, GPSEPHREC* BDSEph, SATPVT* Mid)
2. {
3.     // 根据卫星系统类型选择对应星历数据
4.     if (Sys == GPS)
5.     {
6.         EPH = GPSEph + Prn - 1; // GPS 星历数组索引
7.     }
8.     else if (Sys == BDS)

```

```
9.      {
10.          EPH = BDSEph + Prn - 1; // BDS 星历数组索引
11.          T.Week -= 1356;
12.          T.SecOfWeek -= 14;
13.          LT = 3900.0;           // BDS 时间有效性阈值
14.      }
15.      else return false;        // 非 GPS 或 BDS
16. }
```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-3-3 过期星历判断的函数说明表格

函数	内容
名称	CompSatClkOff
功能	过期星历判断
输入	卫星号、系统、GPSTIME、GPSEPHREC *2
输出	SATPVT 结构体

2.4 误差改正

2.4.1 大气误差

2.4.1.1 对流层延迟

1. 数学原理

$$\Delta_{trop} = \Delta_d + \Delta_w = \frac{K_d \cdot p}{\sin\left(\sqrt{E^2 + 6.25}\right) + 2.25} + \frac{K_w \cdot e}{\sin\left(\sqrt{E^2 + 2.25}\right)}$$

$$K_d = 155.2 \times 10^{-7} \cdot \frac{p}{T} (h_d - H)$$

$$K_d = 155.2 \times 10^{-7} \cdot \frac{4810}{T^2} e (h_w - H)$$

$$h_d = 40136 + 148.72 (T_0 - 273.16)$$

$$h_w = 11000$$

$$e = RH \cdot \exp(-37.2465 + 0.213166T - 0.000256908T^2)$$

$$T = T_0 - 0.0065 \cdot (H - H_0)$$

$$p = p_0 \cdot (1 - 0.0000226 \cdot (H - H_0))^{5.225}$$

$$RH = RH_0 \cdot \exp(-0.0006396 (H - H_0))$$

其中 H_0 为参考面的高度， T_0 、 p_0 和 RH_0 分别为参考面上的干温、气压和相对湿度； H 为测站高度， T 、 p 和 RH 分别为测站上的干温、气压和相对湿度； E 为卫星相对于测站的高度角。

2. C++实现:

```
1. double Hopfield(const double H, const double Elev)
2. {
3.     // 容错处理：如果测站高于对流层顶，返回0
4.     .....
5.     // 计算准备参数
6.     .....
7.     // 计算delta
8.     .....
9.     // 返回对流层改正值
10. return delta_d + delta_w;
11. }
```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-4-1 对流层延迟误差计算的函数说明表格

函数	内容
名称	Hopfield
功能	计算对流层延迟误差
输入	接收机大地高，卫星高度角
输出	对流层延迟误差

2.4.1.2 电离层延迟

由于本项目使用的是双频观测值，电离层延迟已经被很好地消除，所以此处不叙述其计算公式以及实现方式。

2.4.2 粗差探测

1. 数学原理

$$\text{MW组合} \quad L_{MW} = \frac{1}{f_1 - f_2} (f_1 L_1 - f_2 L_2) - \frac{1}{f_1 + f_2} (f_1 P_1 + f_2 P_2)$$

$$\text{GF组合} \quad L_{GF} = L_1 - L_2 = \lambda_1 \varphi_1 - \lambda_2 \varphi_2$$

$$|L_{GF}(t_k) - L_{GF}(t_{k-1})| < \delta_{GF} = 0.05\text{m}$$

$$|L_{MW}(t_k) - \bar{L}_{MW}(t_{k-1})| < \delta_{MW} = 3.00\text{m}$$

若满足条件，即没有伪距粗差和相位周跳，计算 MW 组合的平滑值，将观测值设置为可用。否则将观测值设置为不可用，重置 L_{GF} 和 L_{MW} 平滑值。

$$\bar{L}_{MW}(t_k) = \frac{(k-1) \times \bar{L}_{MW}(t_{k-1}) + L_{MW}(t_k)}{k}$$

2. C++实现：

```

1. void DetectOutlier(EPOCHOBS* Obs)
2. {
3.     // 当前历元 GF 和 MW 临时结算结果
4.     for (int i = 0; i < Obs->SatNum; i++)
5.     {

```

```
6.      // 1、检查该卫星的双频伪距和相位数据是否有效和完整，若不全或为0，将Valid 标记为
      false, continue
7.      // 2、计算当前历元该卫星的GF 和MW 组合值
8.      // 存储当前MWGF 计算值
9.
10.     // 3、对于可用的观测数据，计算伪距的IF 组合观测值，用于SPP
11. for (int j = 0; j < MAXCHANNEL; j++)
12. {
13.     // 4、从上个历元的MWGF 数据中查找该卫星的GF 和MW 组合值
14.     if (Obs->ComObs[j].Prn == CurComObs[i].Prn && Obs->ComObs[j].Sys == CurComObs
        [i].Sys)
15.     {
16.         // 5、6、计算当前历元该卫星 GF 与上一历元对应 GF 的差值 dGF，计算当前历元该卫星与上一
        历元对应的平滑值的差值 dMW
17.         // 7、检查 dGF 和 dMW 是否超限，限差阈值建议为5cm 和3m。若超限，标记为粗差，将Valid
        标记为false ，若不超过限，标记为可用将Valid 标记为true，并计算该卫星的MW 平滑值
18.     }
19. }
20. }
21. // 8、将缓冲区组合观测值的拷贝到obs 里
22. }
```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-4-2 粗差探测的计算函数说明表格

函数	内容
名称	DetectOulier
功能	观测数据粗差探测
输入	EPOCHOBS 结构体
输出	EPOCHOBS 结构体

2.5 单点定位与测速

实现基本原理：

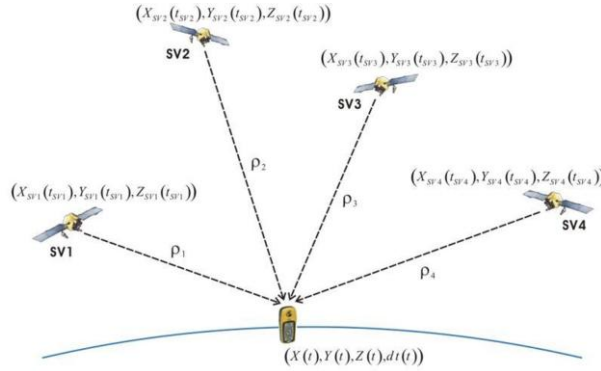


图 2-5-1 单点定位与测速的基本原理图

2.5.1 信号发射时刻卫星状态

1. 数学原理

首先迭代计算某颗卫星 j 的信号发射时刻：

$$t_j^{trans} = t^{recv} - \frac{P_j}{c} - \delta t_j$$

$$\delta t_j = a_{Bias} + a_{Drift} \cdot (t_j^{trans} - t_{toc}) + a_{DriftRate} \cdot (t_j^{trans} - t_{toc})^2$$

其中 t^{recv} 是接收机接收到信号的時刻； P_j 为观测到的伪距； δt_j 是卫星钟差； t_{toc} 是广播星历的参考時刻； $a_{Bias}, a_{Drift}, a_{DriftRate}$ 分别是钟差、钟漂和钟偏；迭代终止条件为： $|\delta t_j^{(k)} - \delta t_j^{(k-1)}| < 10^{-12}$ 。

然后计算卫星位置与速度：

$$\theta = w_e \cdot \frac{\rho_j - \rho_{user}}{c}$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} X^j \\ Y^j \\ Z^j \end{bmatrix}_{ECEF} = R_z(\theta) \cdot \begin{bmatrix} X^j \\ Y^j \\ Z^j \end{bmatrix}_{inertial}$$

$$\begin{bmatrix} V_X^j \\ V_Y^j \\ V_Z^j \end{bmatrix}_{ECEF} = R_z(\theta) \cdot \begin{bmatrix} V_X^j \\ V_Y^j \\ V_Z^j \end{bmatrix}_{inertial}$$

其中 ρ 是卫星与接收机之间的几何距离； w_e 是地球自转角速度； r_{user} 是用户接收机位置；*inertial* 是原先计算的卫星的位置、速度。

接着计算卫星高度角与方位角，并进行对流层改正。此处计算方法前文已经介绍，详细公式可见：[\[见上方（高度角方位角）\]](#) 与 [\[见上方（对流层延迟）\]](#)。

2. C++实现：

```

1. void ComputeSatPVTAtSignalTrans(EPOCHOBS* Epk, GPSEPHREC* GPSEph, GPSEPHREC* BDSEph,
   double UserPos[3])
2. {
3.     // 循环对每个星历求卫星位置
4.     for (int i = 0; i < Epk->SatNum; i++)
5.     {
6.         // 星历过期则跳过计算，并标记为false
7.
8.         // 当系统是GPS系统时
9.         if (Epk->SatObs[i].System == GPS)
10.        {
11.            // 1、迭代计算卫星钟差
12.            // 2、计算卫星位置、速度、钟差和钟速（含相对论效应）
13.            // 3、计算信号传播时间
14.            // 4、计算地球自转改正
15.            // 5、计算卫星的高度角与方位角
16.            // 6、计算对流层改正
17.        }
18.        // 当系统是BDS系统时
19.        else if (Epk->SatObs[i].System == BDS)
20.        {
21.            .....
22.        }
23.    }
24.}

```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-5-1 信号发射时刻卫星状态计算的函数说明表格

函数	内容
名称	ComputeSatPVTAtSignalTrans
功能	计算信号发射时刻卫星状态
输入	EPOCHOBS、GPSEPHREC *2
输出	用户接收机位置

2.5.2 单点定位

1. 数学原理

对于第 j 颗卫星，观测模型为：

$$P_j^{PIF} = \rho_j + c \cdot \delta t_{\text{sys}(j)} - c \cdot \delta t_j + T_j + \delta_{ion}^{PIF} + \varepsilon_j$$

其中 P_j^{PIF} 是 PIF 组合伪距； ρ_j 是卫地几何距离； $\delta t_{\text{sys}(j)}$ 是接收机钟差； δt_j 是卫星钟差； T_j 是对流层改正量； δ_{ion}^{PIF} 是群延迟改正项（BDS 专用 $\delta_{ion}^{PIF} = \frac{f_1^2}{f_1^2 - f_3^2} \cdot c \cdot Tgd_1$ ）； ε_j 是测量噪声项。

接着对方程进行线性化：

$$\begin{aligned} x &= [\Delta x \quad \Delta y \quad \Delta z \quad \Delta(c \cdot \delta t_{GPS}) \quad \Delta(c \cdot \delta t_{BDS})]^T \\ w_j &= P_j^{PIF} - \rho_j - \delta_{corr} \approx B_j \cdot x + v_j \\ B_j &= \begin{bmatrix} -\frac{X_j - X_R}{\rho_j} & -\frac{Y_j - Y_R}{\rho_j} & -\frac{Z_j - Z_R}{\rho_j} & \delta_{j \in GPS} & \delta_{j \in BDS} \end{bmatrix} \end{aligned}$$

其中 x 是状态变量； w_j 观测残差； B_j 是伪距观测方程线性化后的观测矩阵的第 j 行系数向量。

接着进行最小二乘解运算：

$$x = \left(\begin{matrix} B^T & P & B \\ n \times p & n \times n & n \times p \end{matrix} \right)^{-1} \cdot \begin{matrix} B^T & P \\ n \times p & n \times n \end{matrix} \cdot w$$

其中若仅有 GPS 卫星星历时，裁剪 B 的第 5 列，此时列数为 4；若仅有 BDS 卫星星历时，裁剪 B 的第 4 列，此时列数为 4；若同时有 GPS 与 BDS 卫星星历，不裁剪 B 的列数，此时列数为 5。

接着进行定位的迭代更新：

$$\begin{aligned} X^{k+1} &= X^k + x_{1:3} \\ \delta t_{GPS}^{k+1} &= \delta t_{GPS}^k + x_4 \\ \delta t_{BDS}^{k+1} &= \delta t_{BDS}^k + x_5 \end{aligned}$$

其中 X^k 为坐标；收敛判据为： $X^{k+1} - X^k < 10^{-7} \text{ m}$ ；最大迭代次数为 15 次。

接着对定位精度进行评估：

$$\begin{aligned} \sigma_0 &= \sqrt{\frac{(Bx - w)^T P (Bx - w)}{n - p}} \\ PDOP &= \sqrt{q_{xx} + q_{yy} + q_{zz}} \end{aligned}$$

其中 σ_0 是位置精度； $PDOP$ 是位置精度因子。

2. RTK 定位的数据结构体 与 每个历元单点定位结果结构体定义

```
1. struct RAWDAT {
2.     EPOCHOBS BasEpk; // 基准站的原始观测数据
        (EPOCHOBS 结构体)
3.     EPOCHOBS RovEpk; // 流动站的原始观测数据
        (EPOCHOBS 结构体)
4.     SDEPOCHOBS SdObs; // 单差观测数据 (由基准
        站与流动站的原始观测生成)
5.     DDCOBS DDObs; // 双差解算结果 (包含模
        糊度固定信息、基线向量等)
6.     GPSEPHREC GpsEph[MAXGPSNUM]; // GPS 系统的
        广播星历 (每颗卫星一个 GPSEPHREC 结构)
7.     GPSEPHREC BdsEph[MAXBDSNUM]; // BDS 系统的
        广播星历 (每颗卫星一个 GPSEPHREC 结构)
8. };
```

```
1. struct PPRESET
2. {
3.     GPSTIME Time;
        // 当前历元时间
4.     double Position[3];
        // 解算出的 XYZ 坐标 (单位: 米)
5.     double Velocity[3];
        // 解算出的 XYZ 速度 (单位: 米/秒)
```

```
6.     double RcvClkOft[2];
        // 接收机钟差: [0]=GPS 钟差, [1]=BDS 钟差, 单位: 秒
7.     double RcvClkSft;
        // 接收机钟漂 (钟差变化率, 单位: 秒/秒)
8.     double PDOP, SigmaPos, SigmaVel;
        // 定位精度因子、位置与速度的标准差 (单位: 米)
9.     short GPSSatNum, BDSSatNum;
        // 参与解算的 GPS 和 BDS 卫星数
10.    short AllSatNum;
        // 所有参与观测的卫星数 (总数)
11.    bool IsSuccess;
        // 单点定位是否成功 (true=成功)
12.
13.    PPRESET()
14.    {
15.        for (int i = 0; i < 3; i++) Position[i] =
            Velocity[i] = 0.0;
16.        RcvClkOft[0] = RcvClkOft[1] = RcvClkSft
            = 0.0;
17.        PDOP = SigmaPos = SigmaVel = 999.9;
18.        GPSSatNum = BDSSatNum = AllSatNum = 0;
19.        IsSuccess = false;
20.    }
21.};
```

3. C++实现:

```
1. bool SPP(EPOCHOBS* Epoch, RAWDAT* Raw, PPRESET*
    Result)
2. {
3.     // 1、记录当前历元时间
4.     // 2、初始化状态参数
5.     do {
6.         // 3、计算每颗卫星信号发射时刻的卫星位置、钟差等,
            更新 Epoch->SatPVT
7.         // 4、遍历每颗卫星, 构造观测矩阵 B、残差向量 w、权
            阵主对角元素
8.         for (int i = 0; i < Epoch->SatNum; i++) {
9.             // 4.1、无效卫星 (粗差或位置计算失败) 直接跳过
```

```
10.        // 4.2、计算接收机到卫星的空间距离 Rou
11.        // 4.3、构造观测矩阵当前行 (B_row): 前三个元素为
            空间方向余弦, 后两位为系统钟差分量
12.        if (Epoch->SatObs[i].System == GPS) {
13.            // 4.4、GPS 卫星: 第 4 列 (索引 3) 为 1, 其余为 0
14.        }
15.        else if (Epoch->SatObs[i].System == BDS) {
16.            // 4.6、BDS 卫星: 第 5 列 (索引 4) 为 1, 其余为 0
17.        }
18.        // 4.8、观测矩阵、残差向量与权阵主对角同步压入
19.    }
20.    // 5、统计有效卫星总数、解算参数数
```

```
21. // 6、构造观测矩阵、残差向量
22. // 7、构造权阵（等权）
23. // 8、根据参与卫星系统数确定解算参数维数
24. // 仅使用 GPS 卫星
25. if (satnum[0] > 0 && satnum[1] == 0) {
26.     // 清除 BDS 钟差列
27. }
28. // 仅使用 BDS 卫星
29. else if (satnum[1] > 0 && satnum[0] == 0) {
30.     // 构造仅含 XYZ + BDS 钟差的观测矩阵
31. }
32. // GPS+BDS 联合解，参数数为5，直接解
33. else {
34. }
35. // 9、判断收敛条件（参数修正量足够小）

36. // 10、更新参数估计：坐标+系统钟差
37. // 11、计算定位精度（PDOP，取位置参数对应的协方差）
38. // 12、计算验后单位权中误差
39. // 13、迭代计数，超限保护
40. if (count > 15) flag = false;
41. } while (flag); // 14、参数未收敛时继续迭代
42. // 15、输出最终收敛位置结果
43. Result->RcvClkOfst[0] = dt_R0[0];
44. Result->RcvClkOfst[1] = dt_R0[1];
45.
46. Result->Position[0] = X_R0[0];
47. Result->Position[1] = X_R0[1];
48. Result->Position[2] = X_R0[2];
49. }
```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-5-2 单点定位结果计算的函数说明表格

函数	内容
名称	SPP
功能	计算单点定位结果
输入	EPOCHOBS、RAWDAT 结构体
输出	PPRESULT 结构体

2.5.3 单点测速

1. 数学原理

对于第 j 颗卫星：

$$D_j = \dot{\rho}_j + c \cdot \dot{\delta}t_j + \varepsilon_j$$

其中 D_j 是观测多普勒频移； $\dot{\rho}_j$ 是卫星与接收机之间的相对径向速度； $\dot{\delta}t_j$ 是卫星钟漂； c 是光速； ε_j 是观测误差。

接着计算相对速度：

$$\rho_j = \|r_R - r_j\|$$

$$\dot{\rho}_j = -\frac{(r_R - r_j)^T \cdot v_j}{\rho_j}$$

其中 ρ_j 是空间距离； $\dot{\rho}_j$ 是相对速度投影； $r_R = (x, y, z)$ 是接收机的已知位置；

接着构建现行观测模型：

$$x = \begin{bmatrix} V_x & V_y & V_z & c \cdot \dot{\delta} t_R \end{bmatrix}^T$$

$$w_j = D_j - \left(\dot{\rho}_j - c \cdot \dot{\delta} t_j \right)$$

$$B_j = \begin{bmatrix} \frac{x_R - x_j}{\rho_j} & \frac{y_R - y_j}{\rho_j} & \frac{z_R - z_j}{\rho_j} & 1 \end{bmatrix}$$

其中 x 为状态变量； w_j 为每颗卫星的线性方程； B_j 为对应观测矩阵的第 j 行内容。

接着进行最小二乘解算：

$$w = B \cdot x + v$$

$$N = B^T B, \quad W = B^T w$$

$$\hat{x} = N^{-1} \cdot W$$

$$\hat{v}_R = \begin{bmatrix} V_x & V_y & V_z \end{bmatrix}^T = \begin{bmatrix} \hat{x}_{(1)} & \hat{x}_{(2)} & \hat{x}_{(3)} \end{bmatrix}^T, c \cdot \hat{\delta} t_R = x_{(4)}$$

其中 w 是残差向量； N, W 是法方程； $\hat{x}$ 是最小二乘解； $\hat{v}_R$ 是解算速度。

最后计算速度估计的精度，即单位权中误差：

$$\sigma_v = \sqrt{\frac{v^T \cdot v}{n-4}} = \sqrt{\frac{\sum_j (w_j - \hat{w}_j)^2}{n-4}}$$

其中 σ_v 是速度的单位权中误差。

2. C++实现：

```
1. void SPV(EPOCHOBS* Epoch, PPRESULT* Result)
2. {
3.     // 遍历所有卫星，计算每颗卫星的观测矩阵和残差
4.     for (int i = 0; i < Epoch->SatNum; i++)
5.     {
6.         // 1、构造观测矩阵 B 的当前行
7.         // 2、计算残差 w
8.         // 3、记录有效卫星个数
9.     }
```

```
10. // 如果有效卫星数不足 4 个，则退出
11.
12. // 计算法方程矩阵  $N = B^T * B$ 
13. // 计算  $W = B^T * w$ 
14. // 使用行列式判断矩阵是否可逆
15. // 进行最小二乘求解
16. // 计算验后单位权中误差
17.
18. // 将结果保存到 Result 中
19.}
```

上述函数代码为简化代码实现，该函数的功能与说明如下表：

表 2-5-3 单点测速结果的计算函数说明表格

函数	内容
名称	SPV
功能	计算单点测速结果
输入	EPOCHOBS 结构体
输出	PPRESULT 结构体

2.6 获取实时数据流

1. C++实现:

```
1. #pragma comment(lib, "WS2_32.lib")
2. #pragma warning(disable:4996)
3.
4. void CloseSocket(SOCKET& sock)
5. {
6.     closesocket(sock);
7.     WSACleanup();
8. }
```

```
1. bool OpenSocket(SOCKET& sock, const char IP[], c
   onst unsigned short Port)
2. {
3.     WSADATA wsaData;
4.     SOCKADDR_IN addrSrv;
```

```
5.
6.     if (!WSAStartup(MAKEWORD(1, 1), &wsaData))
7.     {
8.         if ((sock = socket(AF_INET, SOCK_STREAM,
9.                             0)) != INVALID_SOCKET)
10.        {
11.            addrSrv.sin_addr.S_un.S_addr = inet_
12.            addr(IP);
13.            addrSrv.sin_family = AF_INET;
14.            addrSrv.sin_port = htons(Port);
15.            connect(sock, (SOCKADDR*)&addrSrv, s
16.            izeof(SOCKADDR));
17.            return true;
18.        }
19.        return false;
20.    }
```

两个函数的功能与说明如下表:

表 2-6- 1 获取实时数据流的函数说明表格

函数	内容 1	内容 2
名称	CloseSocket	OpenSocket
功能	关闭串口通信	打开串口通信
输入	SOCKET 结构体	SOCKET 结构体、IP、端口
输出	/	/

2.7 主函数设计

2.7.1 实现逻辑

本程序支持两种核心工作模式，由用户在程序启动时选择。其中，实时定位模式（`mode==0`）是软件的核心功能之一，其设计旨在通过网络实时接收 GNSS 数据流，并进行连续的 PVT 解算。其详细实现逻辑如下：

1. 初始化与网络配置：

- a) 程序首先初始化核心数据结构体，包括用于存储原始观测数据的 EPOCHOBS、存储广播星历的 RAWDAT、存储单点定位解算结果的 PPRESULT 以及存储参考位置的 POSRES。
- b) 调用 `OpenSocket()` 函数，根据预设的 IP 地址(47.114.134.129)和端口号(7190)建立一个 TCP/IP 套接字连接，用于接收来自远程服务器的实时 GNSS 数据流。若连接失败，则程序提示错误并退出。

2. 实时数据处理循环：

- a) 网络连接成功后，程序进入一个无限循环（`while (true)`），以历元为周期持续处理数据。
- b) 数据接收：在每个循环开始时，程序通过 `recv()` 函数从已建立的套接字中接收二进制数据。为了确保能接收到完整的 NovAtel 数据帧，程序将接收到的数据块持续存入一个主缓冲区 `Buff` 中。
- c) 数据解码：调用核心解码函数 `DecodeNovOem7Dat()` 对主缓冲区中的二进制数据流进行解析。该函数负责查找数据帧头、校验数据完整性（CRC 校验），并根据消息 ID（如 43, 7, 1696, 42）将观测值、广播星历和参考位置等信息解码并填充到对应的 `Obs`、`Eph` 和 `Pos` 结构体中。如果当前缓冲区的数据不足以构成一帧完整的、可供解算的数据，该函数将返回非零值，主循环会跳过本轮解算，继续接收下一批数据。
- d) 粗差探测：解码成功并获得一个历元的完整观测数据后，立即调用 `DetectOutlier()` 函数。该函数利用 MW/GF 组合对双频伪距和载波相位观测值进行检查，探测并标记潜在的粗差和周跳，确保后续解算的可靠性。
- e) PVT 解算：

1.e.1. 调用 `SPP()` 函数，利用经过粗差探测的观测数据和已获取的广播星历，进行标准单点定位解算。该函数通过最小二乘法迭代求解接收机三维坐标和钟差。

1.e.2. 若 SPP() 解算成功, 则紧接着调用 SPV() 函数, 利用多普勒观测值进行单点测速解算, 计算接收机的三维速度。

f) 结果输出: 将 SPP 和 SPV 的解算结果 (位置、速度、钟差、精度因子等) 通过 OutputResult_RealTime() 函数实时输出到控制台窗口, 供用户查看。

2.7.2 C++实现

```

1. // 实时定位模式 (mode == 0)
2. else if (mode == 0)
3. {
4.     // 1. 初始化数据缓冲区和网络套接字
5.     unsigned char Buff;
6.     unsigned char buff;
7.     SOCKET NetGps;
8.
9.     // 2. 打开网络套接字并连接到数据源
10.    if (OpenSocket(NetGps, "47.114.134.129", 719
11.        0) == false)
12.    {
13.        printf("This ip & port was not opened.\n
14.            ");
15.        return 0;
16.    }
17.
18.    // 3. 进入实时处理无限循环
19.    while (true)
20.    {
21.        Sleep(980); // 等待约 1 秒, 以匹配 1Hz 数据更
22.            新率
23.        // 3.1 接收实时数据流
24.        if ((lenR = recv(NetGps, (char*)buff, MA
25.            XRAWLEN, 0)) > 0)
26.        {
27.            memcpy(Buff + lenD, buff, lenR);
28.            lenD = lenR + lenD;
29.            memset(buff, 0, MAXRAWLEN);
30.
31.            // 3.2 解码 NovAtel 二进制数据
32.            // 如果解码函数未返回 0, 说明数据不完
33.                整, 继续接收
34.            if (DecodeNovOem7Dat(Buff, lenD, &Ob
35.                s, Eph.GpsEph, Eph.BdsEph, &Pos) != 0)
36.                continue;
37.
38.            // 3.3 粗差探测
39.            DetectOutlier(&Obs);
40.
41.            // 3.4 进行 SPP 和 SPV 解算
42.            if (SPP(&Obs, &Eph, &Result)) // SP
43.                P 定位解算
44.            {
45.                SPV(&Obs, &Result); // SPV
46.                测速解算
47.            }
48.            else continue; // SPP 失败则跳过
49.
50.            // 3.5 实时输出解算结果
51.            OutputResult_RealTime(&Result, &Pos,
52.                &Obs);
53.        }
54.    }
55.    // 将接收到的数据块存入主缓冲区

```

第三章 结果精度分析

3.1 引言

在本章中，我们将对项目运行过程中采集的定位结果数据进行分析，讨论并分析数据采集时间段内的定位精度变化规律，评估系统的稳定性和鲁棒性。通过对定位误差数据的深入分析，讨论误差产生的主要原因，进一步加深对定位导航的理解，为后续课程中的进一步系统优化提供理论依据。

3.2 数据说明

3.2.1 数据来源

从网络端实时获取，型号为 OEM719、K708 或 UB480 的 GNSS 板卡中的伪距、多普勒频移、广播星历等数据。



图 3-2- 1 GNSS 板卡 司南 K708（左） 芯星通 UB480（右）

本次报告运行项目并采集了一个时间段的数据做分析，时间段如下：

表 3-2- 1 数据采集时间段说明表格

日期	时间段	总时长
2025 年 7 月 28 日	00:00 ~ 12:00	12 小时

3.2.2 结果格式

进行定位计算得到接收机三维坐标位置后，我们使用解算结果与 BestPos 中播发的精确结果进行对比分析，计算得到 ENU 三个方向的三维误差，并将结果保存在 Result.txt 中，格式如下表：

表 3-2-2 定位结果误差格式说明表格

字段名称	描述	单位
dENU	时间戳（年 + 周 + 周内秒）	/
dE	东向定位误差	米
dN	北向定位误差	米
dU	天向定位误差	米
PDOP	位置精度因子	无量纲
SigmaPos	位置标准差	米
SigmaVel	速度标准差	米/秒
GPS_RcvClkOf	GPS 接收机钟差	秒
BDS_RcvClkOf	BDS 接收机钟差	秒
RcvClkSft	接收机钟漂	秒/秒
GPS_Sats	使用的 GPS 卫星数	个
BDS_Sats	使用的 BDS 卫星数	个
Sum_Sats	使用的卫星总数	个

```

dENU: 2374 472073.00000 dE: 0.27592 dN: -0.50465 dU: 1.07910 PDOP: 1.60242 SigmaPos: 1.05663 SigmaVel: 0.02315 GPS_RcvClkOf: -2.03587 BDS_RcvClkOf: 0.00000 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 0 Sum_Sats: 9
dENU: 2374 472074.00000 dE: 0.50307 dN: -0.35828 dU: 0.51797 PDOP: 1.60232 SigmaPos: 1.06702 SigmaVel: 0.01060 GPS_RcvClkOf: -2.53307 BDS_RcvClkOf: 0.00000 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 0 Sum_Sats: 9
dENU: 2374 472075.00000 dE: 0.54212 dN: 1.15377 dU: 2.06902 PDOP: 0.99744 SigmaPos: 1.23892 SigmaVel: 0.02435 GPS_RcvClkOf: -1.59904 BDS_RcvClkOf: 3.45166 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472076.00000 dE: 0.47120 dN: 1.09674 dU: 2.24938 PDOP: 0.99743 SigmaPos: 1.24231 SigmaVel: 0.01879 GPS_RcvClkOf: -1.45434 BDS_RcvClkOf: 3.54745 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472077.00000 dE: 0.56059 dN: 1.14820 dU: 2.04395 PDOP: 0.99742 SigmaPos: 1.24378 SigmaVel: 0.02997 GPS_RcvClkOf: -1.67273 BDS_RcvClkOf: 3.34235 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472078.00000 dE: 0.62081 dN: 1.19185 dU: 1.90420 PDOP: 0.99741 SigmaPos: 1.24905 SigmaVel: 0.01854 GPS_RcvClkOf: -1.87274 BDS_RcvClkOf: 3.14895 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472079.00000 dE: 0.69836 dN: 1.26630 dU: 1.67765 PDOP: 0.99740 SigmaPos: 1.26373 SigmaVel: 0.02164 GPS_RcvClkOf: -2.11490 BDS_RcvClkOf: 2.92771 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472080.00000 dE: 0.68887 dN: 1.27303 dU: 1.69900 PDOP: 0.99740 SigmaPos: 1.26373 SigmaVel: 0.02227 GPS_RcvClkOf: -2.03939 BDS_RcvClkOf: 2.98972 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472081.00000 dE: 0.66600 dN: 1.27748 dU: 1.72297 PDOP: 0.99739 SigmaPos: 1.25827 SigmaVel: 0.02253 GPS_RcvClkOf: -1.92124 BDS_RcvClkOf: 3.08067 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472082.00000 dE: 0.56487 dN: 1.21878 dU: 2.00066 PDOP: 0.99738 SigmaPos: 1.24856 SigmaVel: 0.01739 GPS_RcvClkOf: -1.57358 BDS_RcvClkOf: 3.37544 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472083.00000 dE: 0.40940 dN: 1.12429 dU: 2.40425 PDOP: 0.99737 SigmaPos: 1.26924 SigmaVel: 0.03038 GPS_RcvClkOf: -1.16180 BDS_RcvClkOf: 3.71379 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472084.00000 dE: 0.42827 dN: 1.13464 dU: 2.34972 PDOP: 0.99736 SigmaPos: 1.26918 SigmaVel: 0.02297 GPS_RcvClkOf: -1.23426 BDS_RcvClkOf: 3.63442 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472085.00000 dE: 0.53450 dN: 1.20833 dU: 2.07309 PDOP: 0.99735 SigmaPos: 1.26129 SigmaVel: 0.01766 GPS_RcvClkOf: -1.54925 BDS_RcvClkOf: 3.33891 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472086.00000 dE: 0.47343 dN: 1.16828 dU: 2.22836 PDOP: 0.99734 SigmaPos: 1.26810 SigmaVel: 0.02297 GPS_RcvClkOf: -1.44558 BDS_RcvClkOf: 3.42564 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29
dENU: 2374 472087.00000 dE: 0.42996 dN: 1.11703 dU: 2.37395 PDOP: 0.99734 SigmaPos: 1.28508 SigmaVel: 0.02693 GPS_RcvClkOf: -1.34651 BDS_RcvClkOf: 3.49366 RcvSft: 0.00000 GPS_Sats: 9 BDS_Sats: 20 Sum_Sats: 29

```

图 3-2-2 示例数据格式

3.3 结果分析

3.3.1 结果展示

1. 定位误差

将实时定位结果与 BestPos 解码出的定位结果进行比较计算 ENU(东向、北向、天向)定位误差，即使用定位误差函数 CompEnudPos() 计算 dE、dN、dU，最后计算三维定位误差，结果如下图：

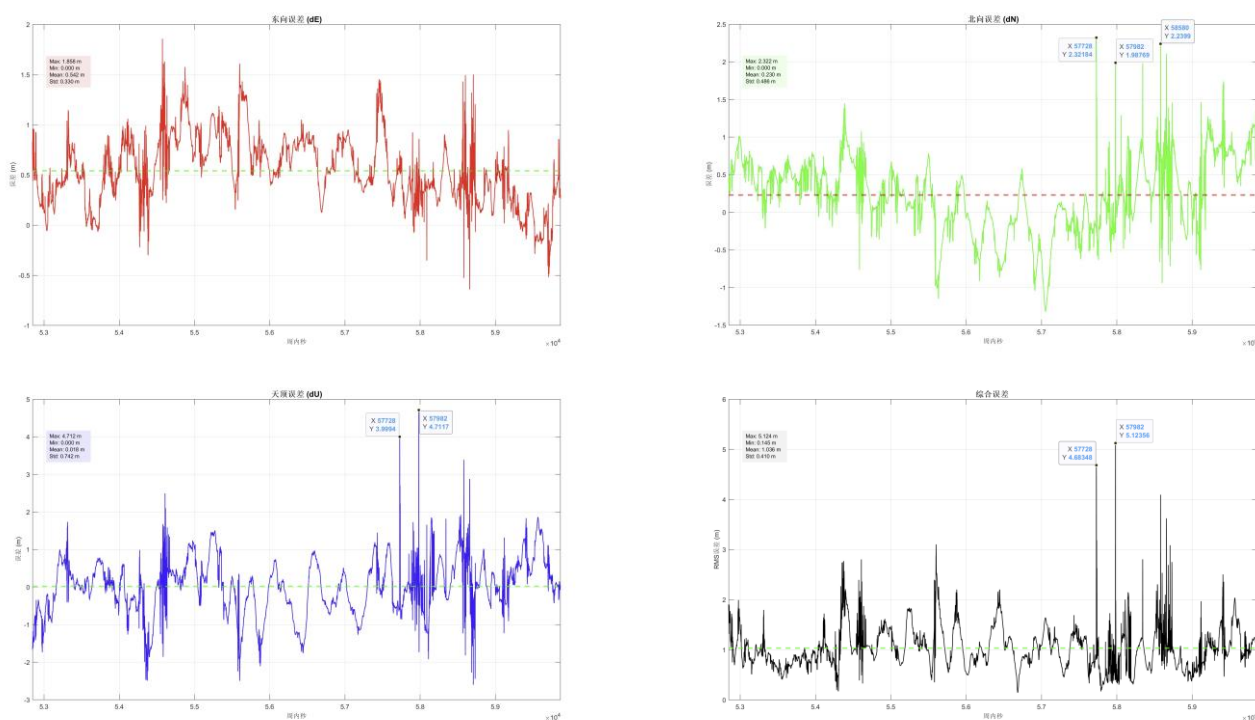


图 3-3- 1 12 小时东向（左上） 北向（右上） 天向（左下） 综合（右下）定位误差图

表 3-3- 1 12 小时定位误差统计表格

属性	东向误差 (m)	北向误差 (m)	天向误差 (m)	综合误差 (m)
最小值	+0.000	+0.000	+0.000	+0.145
最大值	+1.858	+2.322	+4.712	+5.124
平均值	+0.542	+0.230	+0.018	+1.036
标准差	+0.330	+0.486	+0.742	+0.410

可以从图和表格中明显看出，本软件的 12 个小时的综合定位误差为 1.036 米，定位误差控制在 1 米级左右，定位效果较好，但仍有改进的空间。其中东向定位误差平均值最大，达到了 0.542 米，定位误差大的原因极有可能是仍受地球自转的影响；北向定位误差平均值第二，达到了 0.230 米；天

向定位误差平均值最小，为 0.018 米。然而天向的误差标准差最大，为 0.742 米，这表明该方向的定位误差变化波动较大，其中在周内秒 57982 处的定位误差达到 4.7117 米，对综合定位误差造成了严重的影响。

2. 定位精度

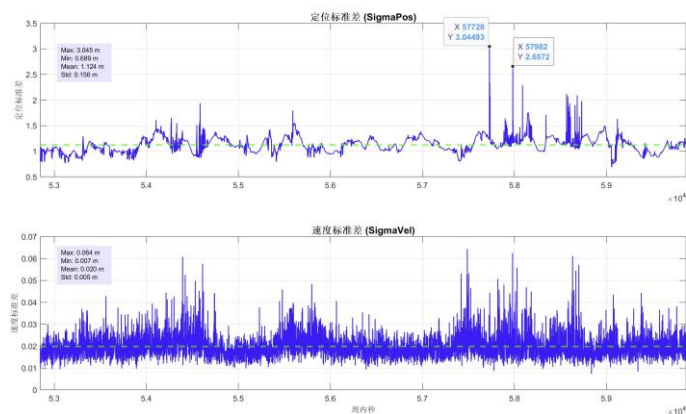


图 3-3-2 定位、速度误差标准差图

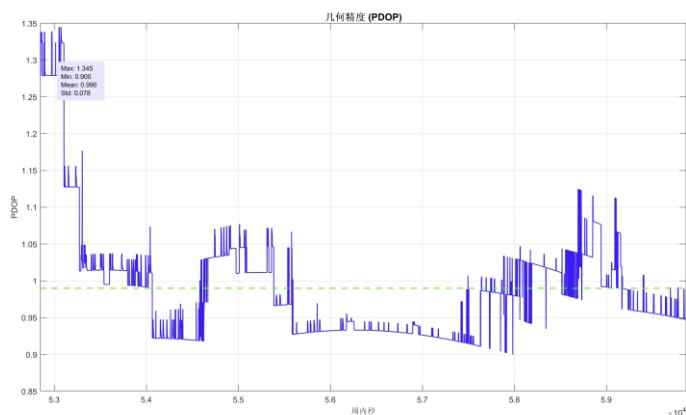


图 3-3-3 卫星几何精度图

表 3-3-2 12 小时定位精度统计表格

属性	SigmaPos (m)	SigmaVel (m/s)	PDOP
最小值	+0.689	+0.007	+0.900
最大值	+3.045	+0.064	+1.345
平均值	+1.124	+0.020	+0.990
标准差	+0.156	+0.005	+0.078

结合前文定位误差和本处图表中可以看出，周内秒 57728 和 57982 处的定位误差标准差最大。整

体位置定位标准差平均值为 1.124 米，速度标准差平均值为 0.020 米/秒，且二者的标准差为 0.156 米和 0.005 米/秒，整体定位效果较好且符合实际静态接收机单点定位。

PDOP 是反应观测卫星几何条件的值，越小越好，认为小于 3.0 的都是较好的观测条件，这里的平均值为 0.990，最大值为 1.345，表明本软件解算的该点观测条件较好，且波动较小，标准差为 0.078。

3. 接收机钟差与钟漂

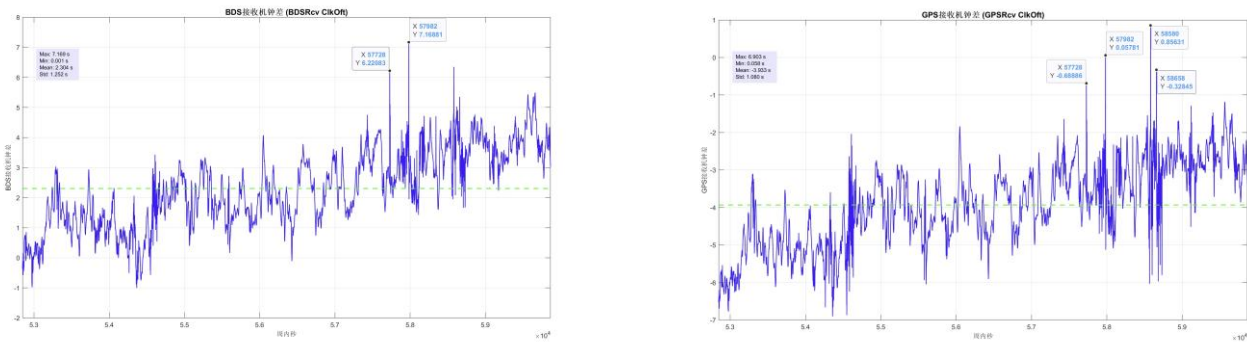


图 3-3- 4 BDS（左） GPS（右）接收机钟差图

表 3-3- 3 12 小时定位接收机钟差与钟漂统计表格

属性	GPS_RcvClkOf (s)	BDS_RcvClkOf (s)	RcvClkSft (s/s)
最小值	+6.903	+7.169	0
最大值	+0.058	+0.001	0
平均值	- 3.933	+2.304	0
标准差	+1.080	+1.252	0

从图表中可以看出 GPS 接收机钟差的平均值为- 3.933 秒，标准差为 1.080 秒；BDS 接收机钟差的平均值为 2.304 秒，标准差为 1.252 秒；本软件并没有对接收机钟漂进行计算。从图中可以明显看出来在周内秒 57728 处和 57982 处的两个系统的接收机钟差明显达到最大值，这与前文的定位误差达到最大值处的周内秒相同，这说明这两处的观测出现了一定的问题，很可能是卫星位置计算出现的问题，而影响卫星位置的计算可能有对流层延迟、多路径误差等。

4. 卫星使用数量

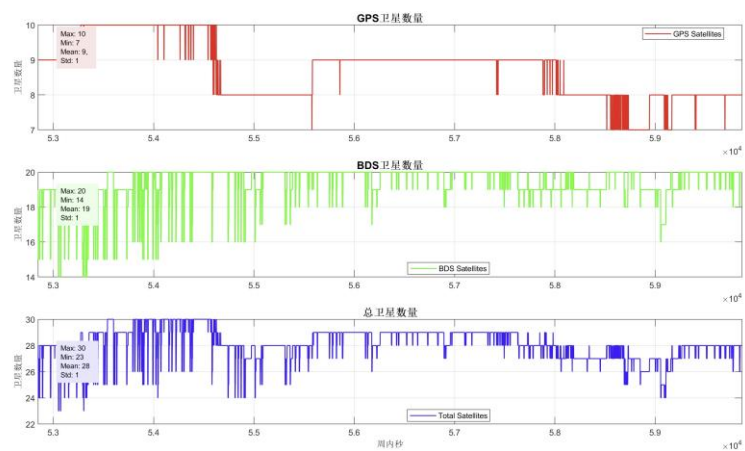


图 3-3-5 卫星使用数量图

表 3-3-4 12 小时定位时卫星使用数量统计表格

属性	GPS_Sats（个）	BDS_Sats（个）	Sum_Sats（个）
最小值	7	14	23
最大值	10	20	30
平均值	9	19	28
标准差	1	1	1

从图表中可以看出，在定位期间，卫星观测总数量平均值为 28，其中 BDS 系统的卫星数量明显大于 GPS 系统的卫星数量，且卫星数量的标准差在 1 个附近，表明本次定位解算的卫星观测数量较多，观测条件较好。

3.3.2 结果分析

对本次为期 12 小时的静态标准单点定位 (SPP) 测试结果进行深入评估, 可以得出结论: 本次课程设计的软件成功实现了 GPS/BDS 双系统双频数据的实时解算, 取得了稳定且符合预期的定位性能。结果显示, 综合三维定位误差的均值仅为 1.036 米。

然而, 要进行更深层次的分析, 我们不能仅仅满足于对最终误差数值的评定。而是对误差时间序列中展现出的系统性特征和异常现象的剖析。本次分析的核心论点在于: 由于项目中采用了双频伪距无电离层 (PIF) 组合观测值 (如 2.5.2 节所述), 这已经有效地消除了单频 SPP 中最大的误差源——电离层延迟。因此, 本次实验结果将原本被巨大电离层误差所掩盖的、更为精细的残余误差项, 如对流层延迟、多路径效应、卫星轨道与钟差误差等, 清晰地暴露出来。接下来的分析将逐一解构这些残余误差在最终定位结果中的具体表现形式及其物理成因。

3.3.2.1 定位误差分量的系统性特征分析

从图 3-3-1 的定位误差序列和表 3-3-1 的统计数据中可以观察到一个显著且经典的现象: 天向分量的误差不论是波动幅度还是标准差, 均显著大于东向和北向分量^[3]。这种现象并非偶然, 经过分析, 其背后是卫星几何构型和大气误差传播特性的双重影响。

1. 卫星几何构型对垂直方向定位的影响

GNSS 定位精度的优劣不仅取决于观测值的质量, 还严重依赖于观测卫星与接收机之间所构成的几何图形, 这一影响通常用位置精度因子 (PDOP) 来量化。对于任何地面接收机而言, 所有可观测的卫星都分布在测站地平线以上的上半球空间, 理想卫星配置情况如下图 3-3-6, 然而地平线以下不存在任何卫星信号源。

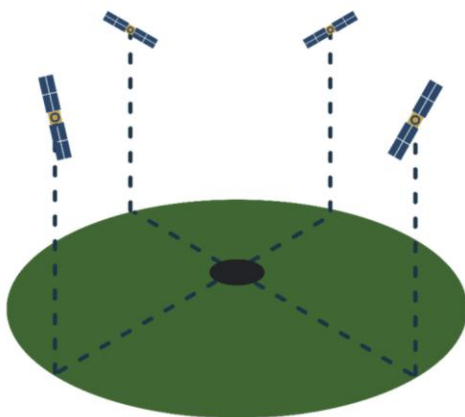


图 3-3-6 理想的卫星配置图

这种只有天顶方向的单侧分布几何结构导致在垂直方向上的约束远弱于水平方向。就像用绳索从不同方向拉住一个物体以固定其位置，如果所有绳索都来自上方，那么该物体在垂直方向上就很容易晃动。同理，GNSS 定位中，垂直精度因子几乎总是大于水平精度因子。尽管本次实验中平均观测卫星数高达 28 颗，确保了优异的整体位置精度因子，但这依然无法弥补垂直方向几何构型上的先天不足。因此，天方向分量天生就对各类观测误差更为敏感，任何微小的测距误差都会在天方向上被放大，从而产生更大的定位误差。

2. 残余大气误差对垂直方向定位的影响

除了几何构型的因素，对流层延迟误差的传播特性进一步加剧了天方向的误差。GNSS 信号穿过对流层时会产生路径延迟，该延迟量与信号路径上的大气密度和水汽含量有关。在解算中，通常先计算天顶方向的总延迟（Zenith Tropospheric Delay, ZTD），然后使用一个映射函数（Mapping Function）将其投影到卫星所在的实际倾斜路径上。这个映射函数是卫星高度角的函数，高度角越低，倾斜路径越长，延迟也越大[4]，具体如下图 3-3-7（右）。

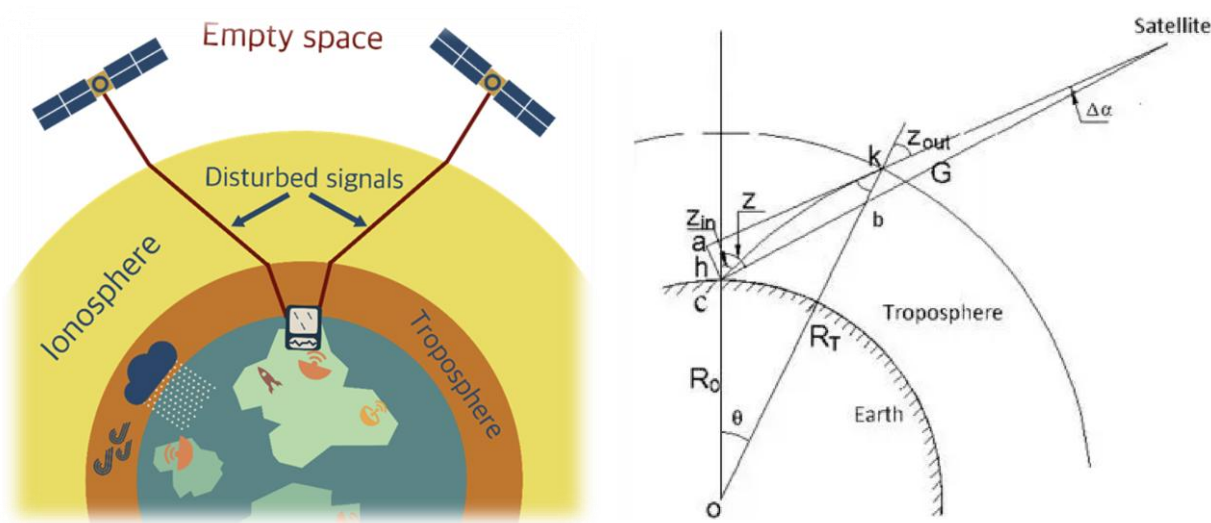


图 3-3-7 对流层延迟原理示意图（左）对流层延迟映射计算示意图（右）

本项目采用了经典的 Hopfield 模型（见 2.4.1.1 节）来估算对流层延迟。Hopfield 模型是一个经验模型，它在没有实时气象参数输入的情况下，依赖标准大气模型来估算延迟。研究表明，这类模型对于占总延迟约 90% 的、由大气压力决定的、变化较为平稳的天顶静力学延迟（Zenith Hydrostatic Delay, ZHD）具有较好的改正效果。然而，对于仅占 10% 但时空变化剧烈、主要由水汽含量决定的天顶湿延迟（Zenith Wet Delay, ZWD），其改正能力则非常有限[4][10][11]。

由于 ZTD 的估算存在残余误差（主要来自 ZWD 部分），这个误差在通过映射函数投影后，其在垂直方向上的分量最大。因此，Hopfield 模型未被精确建模的 ZWD 变化，就直接表现为天方向上持

续的、幅度较大的随机噪声。图 3-3-1 中天方向误差序列图所呈现出的高频抖动和更大的整体标准差，正是这种未被完全消除的对流层湿延迟残差在几何结构最薄弱的垂直方向上被放大的直接体现[5][10]。

3.3.2.2 接收机钟差对定位误差的分析

在本次结果分析中，最具启发性的发现是图 3-3-1（定位误差）和图 3-3-4（接收机钟差）中出现的异常尖峰。在 GPS 周内秒约 57728 和 57982 等时刻，定位误差，尤其是天分量，和接收机钟差估值同时出现了剧烈的、短暂的跳变。这种高度相关性绝非巧合，我们接下来来探索标准单点定位算法在处理某种特定类型误差时的内在机制。

1. 最小二乘误差吸收原理

本次软件中使用的标准单点定位的核心，是基于最小二乘法求解一个包含四个未知数的状态向量： $x = [x_r \ y_r \ z_r \ c \cdot \delta t_r]^T$ ，即接收机的三维坐标和经过光速换算后的接收机钟差。该算法通过调整这四个参数，使得所有卫星的伪距观测值与根据估算参数计算出的理论值之间的残差平方和最小。

在此过程中，状态向量中的各个参数扮演着不同的角色。三维坐标参数 $(x_r \ y_r \ z_r)$ 对每颗卫星的理论距离计算产生的影响是不同的，它与卫星和接收机的相对几何位置有关。然而，接收机钟差项 $c \cdot \delta t_r$ 对于同一历元的所有卫星来说，是一个公共偏差项。

当一个系统性的、影响了多颗卫星的共模误差突然出现在观测值中时，最小二乘算法会寻找最“经济”的方式来吸收这个误差以维持整体残差最小。相比于复杂地调整三个坐标参数去分别拟合每颗卫星的误差，直接调整一个公共参数——接收机钟差，来一次性地、粗略地抵消所有卫星上的共模误差，在数学上是最高效的途径。因此，接收机钟差参数在最小二乘估计中，常常扮演着一个“误差吸收器”或“垃圾桶”的角色，所有未被模型化的、具有相关性的误差，都很容易被错误地归因于它[6][7][8]。

2. 未模型化的多路径效应

基于上述误差吸收原理，可以构建一个完整的因果链来解释观测到的同步尖峰现象，其根本原因极有可能是未被模型化的多路径效应。

- a) **多路径效应：**GNSS 信号在到达接收机天线前，被测站附近的障碍物，如建筑、地面反射，形成了一条或多条非直射路径。这些反射信号与直射信号一同被接收机接收，即为多路径效应。
- b) **观测值污染：**由于反射路径比直射路径更长，多路径信号会产生一个额外的时延，从而污染

伪距测量。这种污染会导致伪距观测值中出现一个大小可达数米、且随卫星→接收机→反射面几何关系快速变化的误差。当多颗卫星在天空中运行时，它们可能在相近的时间段内掠过同一个反射“热区”，导致它们各自的伪距观测值中出现具有相关性的误差。

- c) **误差吸收：**最小二乘解算器在处理这些被污染的观测数据时，探测到了这个作用于多颗卫星的共模误差。如前所述，算法会错误地将这个由多路径引起的伪距偏差归因为接收机钟差的突然变化，从而导致估算出的接收机钟差（RcvClkOft）发生剧烈跳变，以“吸收”这个误差。这正是图 3-3-4 中钟差尖峰的成因。
- d) **误差传播：**一旦接收机钟差被错误地估计，这个被污染的钟差值会反过来代入到最终的位置解算方程中。错误的钟差会导致所有距离计算出现系统性偏差，最终在三维坐标上产生一个显著的定位误差。这解释了图 3-3-1 中与钟差尖峰同步出现的定位误差尖峰。

本项目中实现的基于 MW/GF 组合的粗差探测算法（见 2.4.2 节），对于探测周跳和孤立的大幅值观测异常非常有效。但是，它对于识别和处理更为微妙、具有相关性的多路径误差则能力有限^{[6][7]}。软件中并未包含专门的多路径抑制策略，这使得定位解算过程极易受到此类多路径事件的冲击，从而产生观测到的异常结果。

3.3.2.3 其他残余误差源评估

除了上述主导性的系统误差外，还有一些其他的误差源共同构成了定位结果中的背景噪声，使得误差序列图呈现出持续的随机波动。

- a) **卫星轨道与钟差误差：**本项目使用的是通过导航电文实时播发的广播星历。广播星历提供的卫星位置和时钟信息并非完全精确，其轨道误差的典型值约为 2.5 米，卫星钟差误差约为 2.0 米^{[1][2]}。这些残余误差在不同卫星间通常是不相关的，它们是构成最终定位结果随机噪声的重要组成部分。
- b) **接收机噪声：**任何电子设备都存在内部噪声。GNSS 接收机在信号处理过程中引入的噪声通常会造成约 0.3 米至 1.0 米的测距误差^{[1][2]}。这种噪声是系统固有的，表现为误差时间序列图上高频的、毛刺状的波动，构成了定位结果的最终噪声基底。

总而言之，卫星轨道/钟差的残余误差和接收机噪声共同决定了定位误差的整体“背景”水平，而前述的对流层和多路径效应则是在这个背景之上叠加的、具有明显系统性特征的、幅度更大的误差。

3.3.2.4 误差分析总结与改进

综合以上分析，可以对本次 SPP 实验结果的误差来源进行一个系统性的总结。该软件实现的 1 米级定位精度符合当前技术水平下的预期。误差序列中呈现出的主要特征及其成因如下：

- a) **天向标准差大：**这是由卫星几何构型的固有弱点与本项目采用的 Hopfield 对流层模型对湿延迟改正能力不足两个因素共同作用的结果。
- b) **定位与钟差同步尖峰：**这是由未被模型化的多路径效应污染了伪距观测值，并通过最小二乘算法的误差吸收机制，被错误地归因于接收机钟差，进而污染了位置解。

为了更清晰地展示这一结论，下表对本项目中的主要误差源、抑制方法及残余影响进行了归纳。

表 3-3-5 本项目 SPP 误差源分析综合表格

误差源	量级	抑制方法	抑制效果
电离层延迟	5 ~ 50 米	双频 PIF 组合	完全消除
对流层延迟	2 ~ 20 米	Hopfield 模型	部分抑制
卫星几何构型	/	/	系统性影响
多路径效应	1 ~ 10 米	高度角截止	没有抑制
卫星轨道与钟差	约 2.5 米	广播星历改正	部分抑制
接收机噪声	0.3 ~ 1 米	/	随机性影响

基于以上分析，为进一步提升本软件的定位精度与可靠性，提出以下改进建议：

- a) **升级对流层改正模型：**将 Hopfield 模型替换为更先进的经验模型，如 UNB3m 或 GPT2w/GPT3 模型。这些模型内置了更精细的全球气象参数格网数据，能提供更准确的 ZTD 估值，尤其是在缺乏实测气象数据的情况下^{[5][10]}。此项改进有望显著降低天方向的误差和噪声水平。
- b) **引入多路径抑制策略：**建议在数据处理层面增加专门的多路径抑制算法。一种实用且有效的方法是基于信噪比（SNR）或载噪比（C/N0）的随机模型加权。通常，受多路径影响的信号其 SNR 会降低，因此可以给 SNR 较低的卫星观测值赋予较小的权重，从而降低其在最小二乘解算中的影响^[9]。这将直接有效地抑制观测到的误差尖峰。
- c) **优化随机模型：**可以从目前采用的等权模型（单位阵）转变为更精细化的随机模型。例如，可以引入一个与卫星高度角相关的加权函数，因为低高度角卫星的信号路径更长，更容易受到大气和多路径效应的影响，理应赋予更低的权重^{[4][9]}。这能全面提升解算的稳健性。

第四章 体会与心得

《卫星导航算法与程序设计》这门课程为我提供了从理论到实践的深入学习机会。通过这次学习，我不仅深入理解了 GNSS 技术的复杂性，还体会到将理论中的数学原理转化为实际应用的代码并非易事。实际编程的过程中，我逐渐意识到，只有通过不断的尝试、调试和思考，才能将抽象的理论问题解决并付诸实践。这一过程让我对理论与实践的结合有了更深刻的理解，同时在编程方法、算法设计和数据处理等多个方面积累了宝贵的经验。

课程加深了我对卫星导航原理的理解。首先，由于本次编程课程与卫星导航原理课程同步进行，二者相互辅助、相得益彰，许多内容都能够相互结合并加深理解。这种融合让我更加透彻地掌握了数学公式与原理。其次，编程实现的过程对我们理解原理提出了更高的要求，只有完全掌握数学原理，才能高效、无误地编写高质量的程序。否则，程序的计算结果就会不准确，甚至频繁出现错误，进而导致程序设计过程的无法推进。第三，课程加强了我对卫星观测数据、星历、系统特点、数字敏感度、结果分析和评定的能力。在课程中，我不仅熟练掌握了 NoVATel 数据格式，理解了卫星发射星历的具体内容，还深入了解了 GPS 与 BDS 两个系统的特点和实现过程，并能够准确区分这两者的差异。我对许多量的数字敏感度也有了大幅提升，例如卫星轨道高度、卫星运行速度、对流层延迟大小和卫星接收机钟差等。

课程提升了我在编程算法设计方面的能力。首先，课程让我意识到，最大的区别在于对编程格式规范和可读性的严格要求。我学习到，写出一个整洁、规范且可读性强的程序，不仅体现了程序员对工作的认真负责，也展现了其专业能力。在这门课程的影响下，我在其他课程，如最优估计、卫星导航原理等的程序编写中，开始要求自己更加注重程序的规范性。虽然我之前认为自己做得已经很不错，但最后在与老师的交流中，我才发现还有很多地方可以进一步提升，这对我理解编程语言和编程逻辑起到了非常大的帮助。其次，课程极大地提高了我调试程序的能力。从最初遇到问题时的束手无策，到后来的逐步调试、发现问题所在的过程，这让我体会到了编程中的乐趣。通过这个过程积累的工程经验，不仅让我在后续的程序设计中得心应手，还在其他课程中为我节省了大量时间。第三，课程加强了 my 理论与实践结合的能力，即将数学公式转化为代码的实现能力。从结构体的设计，到函数的输入输出定义，再到嵌套、循环和条件判断的灵活应用，我都获得了极大的提升。这不仅有助于增强程序的可读性，也为后续的程序调试打下了良好的基础，使得后期的工作更加顺利。

最后，我对课程的评价与建议。通过这门课程，我更加明确了理想的教学方式——老师制定方向与方法，学生自主实现，并在过程中积极与老师交流，发现自己的问题。在这个过程中，我们始终有

一个明确的任务导向，在实现的过程中不仅加深了我们对专业原理的理解，还大大提升了我们的工程经验，这对个人的成长有着深远的影响。而在这个过程中积累的工程经验，也为我们其他课程的学习、科研工作以及比赛等活动提供了极大的帮助。总的来说，这门课程让我受益匪浅，已经超出了我的预期，甚至成为了我在专业学习上的一个标杆。我没有任何建议，课程已经做得非常出色，强烈建议其他课程可以从中借鉴。能够遇到您这样一位优秀的老师，是我们的荣幸。再次感谢王老师的无私指导，祝老师身体健康、工作顺利！

参考文献

- [1] VectorNav. (n.d.). GNSS Error Budget.
- [2] NovAtel. (n.d.). An Introduction to GNSS: Chapter 4: GNSS error sources.
- [3] The Pennsylvania State University. (n.d.). GEOG 862: GPS and GNSS for Geospatial Professionals.
- [4] Yao, Y., et al. (2022). A Comprehensive Assessment of the Tropospheric Delay Inversion from ERA5, MERRA2, and CRA40 Reanalysis Data. *Remote Sensing*.
- [5] Wang, N., et al. (2023). Accuracy Analysis of the GPT3 Model and Four Tropospheric Delay Models in China. *Atmosphere*.
- [6] He, C., et al. (2024). An innovation-based cycle slip/multipath estimation, detection, and mitigation method for GNSS/INS/Vision tightly coupled system. *arXiv*.
- [7] Teunissen, P. J. G. (1997). The geometry-free ambiguity success rate. *Journal of Geodesy*.
- [8] Medina, D., et al. (2018). Determination of Multipath Error Models for Multi-Frequency GPS and Galileo Signals. *Proceedings of the 2018 ION ITM Conference*.
- [9] Li, X., et al. (2024). Reduction of Multipath Effect in GNSS Positioning by Applying Pseudorange Acceleration as Weight. *Sensors*.
- [10] Chen, X., et al. (2021). Evaluation of Tropospheric Delay Calculated by Different Models with or without Meteorological Data. *Remote Sensing*.
- [11] Kim, D. & Park, K. D. (2020). Accuracy Comparison of GPT and SBAS Troposphere Models for GNSS Data Processing. *Journal of Positioning, Navigation, and Timing*.

评分项	具体要求	比例	得分
平时成绩	1. 上课认真听课，无缺席、旷课 2. 按时提交模块程序，完成质量较好 3. QQ 群中积极提问，讨论问题	10%	
程序面试	1. 程序模块设计合理性 2. 程序编码规范，注释详细 3. 程序运行结果正常合理 4. 面试中能正确回答老师提出的问题	40%	
总结报告	1. 程序算法总结详细 2. 程序总体设计、模块设计合理 3. 软件测试结果与精度分析 4. 编程体会情况总结深刻	50%	
总分			
评 语：			